GFAz: Order-of-Magnitude Pangenome Analytics by Computing over Compression

Taolue Yang
taolue.yang@temple.edu
Temple University
Philadelphia, PA, USA

Youyuan Liu
youyuan.liu@temple.edu
Temple University
Philadelphia, PA, USA

Bo Jiang
bo.jiang@temple.edu
Temple University
Philadelphia, PA, USA

Xinghua Shi
mindyshi@temple.edu
Temple University
Philadelphia, PA, USA

Sian Jin
sian.jin@temple.edu
Temple University
Philadelphia, PA, USA

Abstract

Pangenome graphs stored in the Graphical Fragment Assembly (GFA) format can reach hundreds of gigabytes, with traversal records accounting for over 90% of the file size. General-purpose compressors fail to exploit cross-haplotype redundancy in these traversals, and existing specialized tools either sacrifice full semantics or offer limited throughput, while still trailing our method in compression ratio. We present GFAz, a lossless GFA compressor that compresses and decompresses in linear time. GFAz uses iterative 2-mer grammar compression to collapse traversal redundancy, combined with columnar, type-specific encoding of all GFA record types including optional fields. Both compression and decompression are fully parallel, with a CPU backend (OpenMP) and a GPU backend (CUDA/nvComp). Across six datasets ranging from 113 MB to 369 GB, GFAz achieves 18–84× compression, outperforming all evaluated baselines in compression ratio, compression throughput, and decompression throughput. The CPU backend sustains up to 385 MB/s compression and 1.8 GB/s decompression; the GPU backend reaches up to 4.8 GB/s and 9.4 GB/s, respectively.

CCS Concepts

• **Theory of computation** → **Data compression**; • **Applied computing** → *Bioinformatics*; • **Computing methodologies** → *Parallel algorithms*.

Keywords

Biological Data Compression, Graphical Fragment Assembly

ACM Reference Format:
Taolue Yang, Youyuan Liu, Bo Jiang, Xinghua Shi, and Sian Jin. 2026. GFAz: Order-of-Magnitude Pangenome Analytics by Computing over Compression. In *2026 International Conference on Supercomputing (ICS '26), July 06–09, 2026, Belfast, United Kingdom.* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3797905.3807870

ACM ISBN 979-8-4007-2522-7/2026/07
https://doi.org/10.1145/3797905.3807870

1 Introduction

Human genomes vary across individuals at many scales, from small variants to large structural changes, yet most analysis still relies predominantly on mapping sequencing reads to a single linear reference genome. While effective in conserved regions, this paradigm breaks down in highly polymorphic loci and in sequences absent from the reference, leading to systematic reference bias [15]. Pangenome graphs [3, 20] address this limitation by representing population variation explicitly within a unified structure: nodes encode DNA segments, edges capture admissible adjacencies, and paths represent individual haplotypes. By eliminating reliance on a single reference, graph-based representations substantially improve downstream analyses, including up to 34% fewer small-variant errors and more than doubling structural-variant discovery compared to linear-reference approaches [14].

Recently, the Graphical Fragment Assembly (GFA) format [6] has emerged as the de facto interchange format for pangenome graphs. GFA is simple, human-readable, and widely adopted by graph construction and analysis pipelines such as Minigraph-Cactus [10] and PGGB [4]. However, this generality of GFA comes at a cost of scale. While graph topology grows sublinearly as haplotypes are added, traversal records grow nearly linearly because each traversal is materialized explicitly. This structural redundancy rapidly dominates file size, making large GFA files expensive to store, transfer, and process, and thus motivating the need for effective file compression.

Real pangenome datasets make this bottleneck explicit. Sirén and Paten [23] report that 90-haplotype human pangenome graphs constructed from year-1 HPRC data require 44.9 GiB and 88.6 GiB as uncompressed GFA text for Cactus and PGGB, respectively; even after gzip compression, they remain 11.1 GiB and 14.6 GiB. At larger cohort scale, their 5008-haplotype 1000GP graph reaches 9534.9 GiB uncompressed and 2231.3 GiB after gzip compression. This growth is largely driven by traversal data: in a human chromosome 19 graph with 1,000 haplotypes, traversals alone account for 98.7% of a 14 GB GFA file [8].

In practice, gzip-compatible compressors such as gzip and bgzip remain widely used in bioinformatics pipelines because they are ubiquitous, stable, and interoperable with existing Unix tooling. However, pangenome graphs expose a mismatch between general-purpose compression and graph structure. Gzip-style compressors operate over local byte windows and are not designed to exploit repeated traversal structure across chromosome-scale haplotypes, leaving substantial cross-haplotype redundancy unexploited.

Two specialized approaches illustrate the current design trade-offs. GBZ [23] achieves weaker compression on highly collapsed regions and small graphs, and is not designed for incremental updates. In contrast, sqz [8] preserves a text workflow but is relatively slow in practice due to its heap-based Byte Pair Encoding (BPE) design. Both approaches do not preserve all original GFA record information, such as optional fields, which limits their use as fully lossless GFA text compressors.

We thus present GFAz, a GFA compressor that achieves state-of-the-art compression ratio and high throughput on both CPU and GPU. In summary, GFAz makes four main contributions:

- **Orientation-aware grammar compression**: multi-round 2-mer grammar that exploits the canonical symmetry of forward/reverse node orientations with linear-time compression and decompression.
- **Columnar GFA representation**: column-oriented layout for all GFA record types with type-specific encodings, enabling individual field access without full decompression.
- **Incremental haplotype extension and random access**: support for adding extra haplotypes without recompressing previously compressed data, and for retrieving individual haplotypes without decompressing the full dataset.
- **Fully parallel CPU and GPU backends**: end-to-end parallel compression and decompression using OpenMP and CUDA, with a shared format so that data compressed on CPU can be decompressed on GPU for faster decoding.

Across large human pangenome datasets, GFAz achieves 18–84× compression, typically over 10× better than gzip/zstd baselines. CPU throughput reaches 200–400 MB/s for compression and 2 GB/s for decompression; the GPU backend reaches up to 5 GB/s and 10 GB/s, respectively. These results make GFAz practical for large-scale pangenome storage and transfer. We release our implementation as open source at: https://github.com/babyplutokurt/GFAz.

2 Background

2.1 The GFA Format

GFA is a line-oriented text format for sequence graphs [6]. Biologically, a graph node represents a DNA sequence segment (often shared by many individuals), edges represent allowed adjacencies between segments, and traversals represent assembled sequences or haplotypes as ordered walks through those segments.

The node record is **S** (segment), which stores a segment identifier and its DNA sequence. Edge records include **L** (link), **J** (jump), and **C** (containment): **L** connects oriented segments (with overlap information), **J** represents long-range connections or gaps, and **C** represents containment/nesting relationships. Traversal records are **P** (path) and **W** (walk). Both describe an ordered, orientation-aware route through nodes, which reconstructs a longer biological sequence from graph segments. **P** is a general named traversal (often a contig or reference path), while **W** (GFA 1.1) represents haplotype walks with explicit sample, haplotype, and coordinate metadata. **H** (header) stores file-level metadata, and optional `TAG:TYPE:VALUE` fields extend records with additional annotations. Figure 1 shows a visualization of a GFA file using Bandage [25].

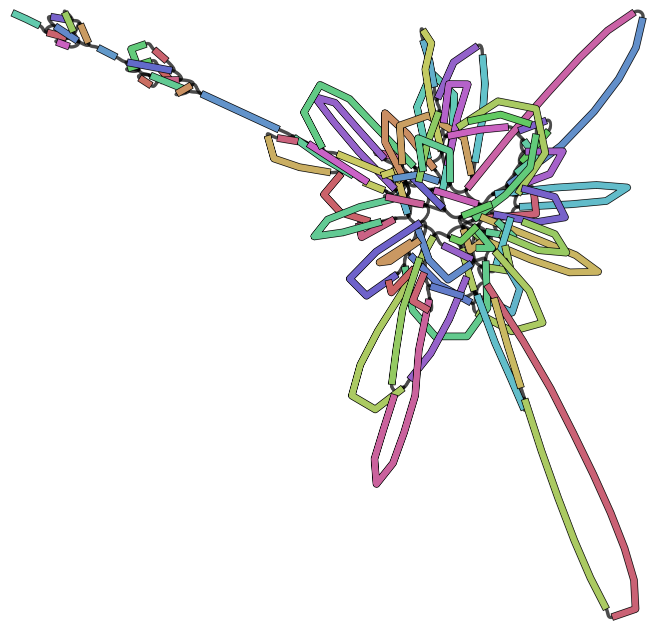


Figure 1: Assembly graph in GFA format, visualized with Bandage [25]. Colored elements denote sequence segments (nodes; S records). Black connections indicate edges between oriented segments, represented by L (link), J (jump), or C (containment) records. Any valid traversal through connected segments corresponds to a P (path) or W (walk) record.

2.2 The Scalability Challenge

GFA is widely used in graph-based genomics, including assembly graphs, pangenome construction, and downstream analysis. In pangenome graphs, shared sequence is merged into common nodes and variants form alternate branches. HPRC releases have grown from tens to hundreds of haplotypes and continue to expand [12, 24]. Many downstream tools operate directly on GFA graphs, so these large files create both memory pressure during analysis and substantial storage overhead. At this scale, GFA becomes a systems bottleneck rather than only a storage nuisance.

As highlighted in Section 1, real pangenome GFA files already reach very large sizes. The core scaling mechanism is structural: topology (S/L/J/C records) grows sublinearly as haplotypes are added, but traversal records (P/W) grow near-linearly because each haplotype walk is materialized explicitly. Since human haplotypes are highly similar, these long traversal streams contain substantial repeated node-ID substrings across samples. At even larger population sequencing scales, this growth will continue, pushing GFA datasets beyond hundreds gigabytes into multi-terabyte regimes. In practice, pipelines also produce many intermediate GFAs (per chromosome, per version, and per region), further amplifying storage and transfer costs.

Generic compressors cannot exploit most of this redundancy. Their small sliding windows miss long-range similarity across chromosome-length paths, so compression ratio remains limited for GFA files.

Table 1: Mapping from GFA record types to GFAz internal integer-based representation.

GFA Record	Graph Role	Internal Representation
S-line	Node Definition	`uint32_t` (1-based ID)
L, J, C-lines	Graph Topology	`vector<int32_t>`
P, W-lines	Graph Traversal	`vector<vector<int32_t>>`

2.3 Existing Compression Approaches

Because generic compressors miss long-range path redundancy, specialized GFA compressors are needed. Other genomic compression families are also not a direct fit: read/alignment formats such as BAM/CRAM [1, 13] target mapped reads rather than graph interchange files, and FASTQ compressors such as GPUFastQLZ [26] target linear sequence collections rather than graph topology and traversal semantics. For GFA graphs, compression must remain fully lossless to preserve graph structure and metadata while still exploiting cross-haplotype traversal redundancy. Two graph-native approaches are GBZ and sqz.

GBZ uses Graph-Based Burrows–Wheeler Transform (GBWT) [21, 22] to store large collections of similar paths efficiently. This design is effective for large pangenomes with many similar haplotypes. Its weaknesses are practical. Performance can degrade in highly collapsed repetitive regions, and compression ratio is less favorable on small graphs. Because GBWT construction is global, GBZ also does not naturally support incremental updates (e.g., appending new haplotypes) without rebuilding the index. In addition, GBZ does not preserve full GFA record semantics, like optional fields.

sqz keeps a text workflow and applies a heap-based BPE grammar design [27] to path records. It repeatedly selects frequent adjacent 2-mers and replaces them with new rules. The main limitations are speed and memory cost. After each new rule, sqz updates frequencies of affected neighboring pairs through adjacency-list bookkeeping, which makes each rule-creation step expensive in practice. This sequential dependency also limits parallelization, because each next substitution depends on the current global pair statistics. Similar to GBZ, sqz does not preserve full optional-field information, so it is not truly lossless for complete GFA records.

3 Methodology

This section presents GFAz as a modularly designed compression pipeline with two interchangeable backends: CPU (OpenMP) and GPU (CUDA/Thrust/nvComp) [17]. The final entropy-encoding layer is also modular, so it can be changed or tuned easily to trade compression ratio against throughput.

3.1 Graph View of GFA

A GFA file is a tab-delimited graph format with a small set of record types. GFAz converts these heterogeneous text records into a compact Structure-of-Arrays layout of typed columns. Segments (S-lines) receive compact 1-based integer IDs, with sequences stored separately. Links, jumps, and containments (L/J/C-lines) become integer endpoint columns plus bit-packed orientation columns, with auxiliary strings stored as concatenated byte streams and offsets. Most importantly, both paths and walks (P/W-lines) are normalized to the same signed `int32_t` traversal representation, where the sign

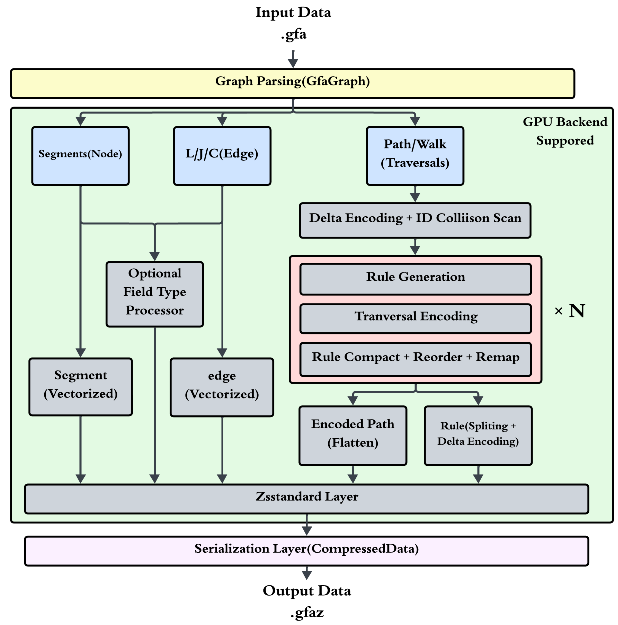


Figure 2: GFAz compression workflow. The grammar compression loop repeats for N rounds (default 8); each round replaces repeated 2-mers, reducing the traversal stream by up to half.

encodes orientation; for example, `>s1<s2>s3` becomes `[1, -2, 3]`. This conversion turns verbose GFA text into compressible integer and byte streams, with traversals forming the dominant input to the grammar-compression pipeline.

Table 1 summarizes how the text-to-integer mapping prepares each component for the compression pipeline.

3.2 Architecture Overview

Figure 2 and 3 show the end-to-end workflow. A GFA file is first parsed into an in-memory graph representation: on the CPU this is a `GfaGraph` with nested vectors; for the GPU backend the traversals are flattened into a single contiguous array paired with a lengths vector, with offsets computed on-demand via parallel prefix sum. The parsed graph is then compressed into a `CompressedData` object containing byte blocks for every field, and can be serialized to a binary `.gfaz` file for persistent storage. Decompression is intentionally asymmetric: fixed-width graph fields are restored from their encoded columns, but traversal records are expanded and emitted as GFA text in a streaming fashion rather than rebuilding a full in-memory `GfaGraph`.

Both compression and decompression are fully parallel at every step. As introduced above, the pipeline supports CPU and GPU execution; the GPU backend keeps traversal data device-resident throughout the compression loop, transferring only the final compressed blocks back to the host.

3.3 Compression Pipeline

The central challenge in GFA compression is traversal data (P and W records), which often exceeds 90% of file size. Traversals are

Algorithm 1 GFAz Compression Pipeline

Require: GFA file F, rounds R, delta_rounds D
Ensure: Compressed data C
$G \leftarrow \text{Parse}(F)$
for $i = 1$ to D **do**
  DeltaTransform(G.paths)
  DeltaTransform(G.walks)
end for
next_id $\leftarrow$ max(MaxAbs(G.paths), $|G$.segments$|$) + 1
for $r = 1$ to R **do**
  rules $\leftarrow$ GenerateRules(G.paths, G.walks)
  EncodePaths(G.paths, G.walks, rules)
  CompactSortRemap(rules, G.paths, G.walks)
end for
$C \leftarrow$ CompressAllFields(G, rules)
return C

highly redundant: many haplotypes share long node-ID substrings across P-lines and W-lines.

GFAz exploits this structure with iterative grammar compression. In each round, we find frequent adjacent 2-mers (initially node IDs), assign each 2-mer a new integer ID, and replace all occurrences. Later rounds can combine original nodes with rule IDs, or combine rule IDs with each other. This hierarchy collapses long repeated patterns into single symbols and sharply reduces traversal length.

Rule Definition. We formally define a *rule* as a new unique integer identifier assigned to a repeated 2-mer. Like segment IDs, rule IDs are signed 32-bit integers where the sign encodes orientation. If a rule r ($r > 0$) represents the 2-mer (u, v), then its negation $-r$ represents the reverse complement 2-mer $(-v, -u)$. This symmetry stems from the fact that traversing a subgraph in reverse flips the order and orientation of all constituents.

$$r \Rightarrow (u, v) \quad \iff \quad -r \Rightarrow (-v, -u) \tag{1}$$

By treating rules as oriented symbols identical in format to node IDs, the compression engine can recursively combine nodes and rules without special cases.

3.3.1 Phase 1: Delta Encoding and Rule Namespace Allocation. We begin traversal compression with a single delta-encoding pass. For a sequence $[n_1, n_2, \ldots, n_k]$, we produce $[n_1, n_2 - n_1, n_3 - n_2, \ldots, n_k - n_{k-1}]$ and then run grammar compression on this transformed stream. This step is effective because pangenome graph builders often assign segment IDs in approximate reference order. Many haplotypes therefore contain long runs of locally consecutive IDs, and one delta pass collapses these runs into short repeated differences. For GFAz, the main benefit is not merely smaller values, but a much smaller adjacent-pair space for rule mining: many distinct 2-mers in the original traversal stream become the same few repeated 2-mers after delta encoding. Table 2 shows this effect on a chromosome-scale graph: one delta pass reduces the first-round rule search space by over 25× and gives the best final size.

We therefore use exactly one delta pass throughout the system. Additional delta rounds do not improve compression ratio, because later rule IDs no longer preserve the sequential locality of original segment IDs.

Table 2: Effect of delta encoding rounds on rule search space and final traversal size for `chr6.pan.gfa` (human chromosome 6 pangenome GFA). All results use 8 grammar-compression rounds.

Delta rounds	Rules (round 1)	Total rules	Final size
0	5,455,923	29,391,077	3.60%
1	201,503	6,124,767	2.88%
2	264,485	6,281,692	2.99%
3	372,451	7,053,633	3.11%

To separate literals from grammar rules, we reserve the rule namespace above all literal values. We set the starting ID to:

$$I_{start} = \max\left(\max_{p \in \mathcal{P}} \max_{v \in p} |v|,\ |S|\right) + 1 \tag{2}$$

where the first term covers all values in the delta-encoded streams and $|S|$ covers valid segment IDs. This guarantees that a single check ($|x| \geq I_{start}$) uniquely identifies a rule.

3.3.2 Phase 2: Hierarchical 2-Mer Grammar Compression. The engine of our pipeline is a multi-round algorithm that recursively replaces frequent 2-mers with the rule IDs defined in the previous section. A *2-mer* in this context is simply an ordered pair of signed integers (u, v), representing adjacent elements in a traversal.

Since GFA paths represent double-stranded DNA, a traversal (u, v) and its reverse complement $(-v, -u)$ should map to the same rule. We therefore normalize every observed 2-mer to a *canonical form* by selecting the lexicographically smaller of its forward and reverse representations.

$$\text{pack}(u, v) = ((int64)u \ll 32) \mid (v \,\&\, \texttt{0xFFFFFFFF}) \tag{3}$$

$$\text{canonical}(u, v) = \min\left(\text{pack}(u, v), \text{pack}(-v, -u)\right) \tag{4}$$

By hashing and counting only these canonical values, occurrences on either strand contribute to the same rule. Each rule therefore corresponds to exactly one canonical 2-mer, while the sign of the emitted rule ID records whether the observed occurrence matches the canonical orientation or its reverse.

Rule Generation. To identify 2-mers that appear at least twice, we use a parallel "two-set" strategy that avoids building a full frequency table. Each thread scans adjacent pairs, inserts first occurrences into a local *seen* set, and promotes second occurrences to a local *repeated* set. The thread-local sets are then reduced into global *seen* and *repeated* sets, so a 2-mer observed once in multiple threads is still promoted to *repeated.* Paths and walks are processed together in this pass, ensuring that a 2-mer appearing once in a path and once in a walk is still detected as repeated. Finally, we assign sequential rule IDs to the unique canonical 2-mers in the global *repeated* set.

Suppose n unique repeated 2-mers are found. Each is assigned a rule ID sequentially: the first receives start_id, the second start_id + 1, and so on up to start_id + $n - 1$. Two structures are built: a hash map from canonical 2-mer to rule ID for $O(1)$ lookup during encoding, and a vector of length n indexed by (rule_id − start_id) for $O(1)$ reverse lookup during later expansion.

Traversal Encoding and Rule Tracking. We encode each traversal by scanning left-to-right, checking every consecutive 2-mer against the rule hash map. For each 2-mer (u, v), we first check if

Table 3: Internal columnar data structures for each GFA record type before final ZSTD compression.

GFA Record	Component Fields	Internal Data Structure
Rules	Dictionary (First/Second)	Two parallel vector<int32_t>
	Layer Ranges	vector<LayerRuleRange>
Segments (S)	Sequences	Concatenated string + Lengths (vector<uint32_t>)
	Optional Fields	Type-specific columns (varint, float, string)
Links (L)	From/To Node IDs	Parallel vector<uint32_t>
	Orientations	Bit-packed vector<uint8_t> (1 bit/edge)
	Overlaps	Parallel vector<uint32_t> / vector<char>
	Optional Fields	Type-specific columns
Jumps (J)	From/To Node IDs	Parallel vector<uint32_t> (delta+varint)
	Orientations	Bit-packed vector<uint8_t>
	Distance	Concatenated string + Lengths
	Rest Fields	Concatenated string + Lengths
Containments (C)	Container/Contained IDs	Parallel vector<uint32_t> (delta+varint)
	Orientations	Bit-packed vector<uint8_t>
	Position	vector<uint32_t> (ZSTD)
	Overlap (CIGAR)	Concatenated string + Lengths
	Rest Fields	Concatenated string + Lengths
Paths (P)	Encoded Sequence	Flattened vector<int32_t> (Rules/IDs)
	Path Names, Overlaps	Concatenated string + Lengths
	Path Lengths	vector<uint32_t> (Original & Compressed)
Walks (W)	Encoded Sequence	Flattened vector<int32_t> (Rules/IDs)
	Sample/Seq IDs	Concatenated string + Lengths
	Haplotype/Start/End	vector<uint32_t> + vector<int64_t> (varint)
	Walk Lengths	vector<uint32_t>

its canonical form corresponds to a known rule ID r. If so, we check orientation: if (u, v) matches the canonical order, we replace the 2-mer with $+r$; if it matches the reverse complement order $(-v, -u)$, we replace it with $-r$.

Because rules are generated from the pre-encoding stream, newly emitted rule IDs cannot participate in additional same-round matches, so encoding remains a simple left-to-right scan.

During this process, we maintain a usage bitmap, `rules_used`, to track which generated rules are actually applied. This is necessary because greedy substitution may leave some generated rules unused; whenever a rule r is emitted, we set `rules_used`$[|r| - \text{start\_id}] = 1$.

Compact, Sort, and Remap. After encoding, we optimize the rule set for storage and the next grammar round via three fused operations:

(1) **Compact (Prune Unused):** As noted, greedy encoding leaves gaps in the rule index space. To remove them, we perform an *exclusive prefix sum* on the `rules_used` bitmap. The result at index i gives the exact destination index for the rule at $(\text{start\_id} + i)$ in the new particular array. For example, if `rules_used` is $[1, 0, 1, 1]$, the prefix sum is $[0, 1, 1, 2]$. This tells us: rule 0 moves to pos 0, rule 1 is skipped, rule 2 moves to pos 1, and rule 3 moves to pos 2. We then gather only the active rules into a value-contiguous array using these computed offsets, completely eliminating gaps in parallel.

(2) **Sort (Optimize Layout):** We sort all the active rules by their packed 2-mer content. This step is crucial for the compression of the rulebook itself. By ordering rules lexicographically (primarily by their first element u, secondarily by v), we cluster rules that share the same starting node. For storage, each packed 64-bit rule key is split into its upper 32 bits and lower 32 bits. After sorting, the upper-32-bit stream becomes highly repetitive (e.g., $[\ldots, u, u, u, w, w, \ldots]$), so delta encoding produces long zero runs and sharply reduces rulebook size.

(3) **Remap (Update Streams):** Finally, we assign permanent, sequential IDs to the rules in their new sorted order, starting from the current global `next_id`. We verify the permutation by building a lookup table, $T[\text{old\_id} - \text{range\_start}] = \text{new\_id}$. A parallel function then sweeps over all paths: any element e falling in the current round's range is updated to $T[|e| - \text{range\_start}]$ (preserving the original sign). Upon completion, we update `start_id` to be the last assigned rule ID +1, ensuring that rules in the next grammar round begin exactly where the current set ends, which results in sequential rule IDs across all rounds, ranging from the first round's start ID to the last end ID.

The Generation-Encoding-Compact, Sort and Remap phases constitute a single round. We repeat this process iteratively (typically 8 times). As described earlier, subsequent rounds can pair the rules created in previous steps, allowing the algorithm to capture exponentially longer patterns (length 4, 8, 16, and beyond) and recursively collapse long repetitive haplotype blocks into single rule IDs. After these rounds, the traversal data consists of encoded traversal streams, a global rulebook stored as parallel `rules_first` and `rules_second` vectors, and the boundary value `start_id` that separates literals from rule IDs during decoding. This representation exposes both traversals and rules as integer streams for the final field-specific encoding phase.

3.3.3 Phase 3: Field-Specific Encoding. After Phases 1–2, GFAz stores the graph as typed streams. Phase 3 converts all records to a columnar Structure-of-Arrays (SoA) layout: one contiguous column per field. Each column then uses a type-specific encoder:

- **Integers (IDs, lengths, positions)**: Near-monotonic ID columns (e.g., link/jump/containment endpoints) use delta + varint. Other fixed-width integer columns use dense arrays + ZSTD. Rule columns use delta + ZSTD.
- **Booleans (orientations)**: Bit-packed (8 values/byte), up to 8× smaller than byte-per-flag storage.
- **Strings (names, DNA, CIGARs, auxiliary fields)**: Concatenated byte streams with offset arrays, then ZSTD.

Optional Fields. (`TAG:TYPE:VALUE`) are encoded by `TYPE`:

- `i` (integer): varint-encoded vector.
- `f` (float): float vector.
- `A` (character): contiguous byte array.
- `Z` (string), `J` (JSON), `H` (hex): concatenated stream with offsets.
- `B` (numeric array): flattened value stream (subtype-dependent, e.g., `c`, `s`, `i`, `f`) + per-row length vector.

Sparse optional fields are handled by sentinel padding: if the N-th record lacks a tag, we insert a sentinel value into that column. This preserves $O(1)$ indexing, and the resulting runs of identical sentinels compress effectively under ZSTD.

Table 3 lists the specific columnar structures used for each GFA component.

3.3.4 Phase 4: ZSTD Compression. All encoded fields are finally compressed with ZSTD [2]. By this stage, the dominant data (traversals) has already been aggressively reduced by grammar compression and type-specific encoding, so ZSTD operates mostly on compact, low-entropy streams. We use ZSTD at level 9 (configurable via environment variable) to balance throughput and compression ratio.

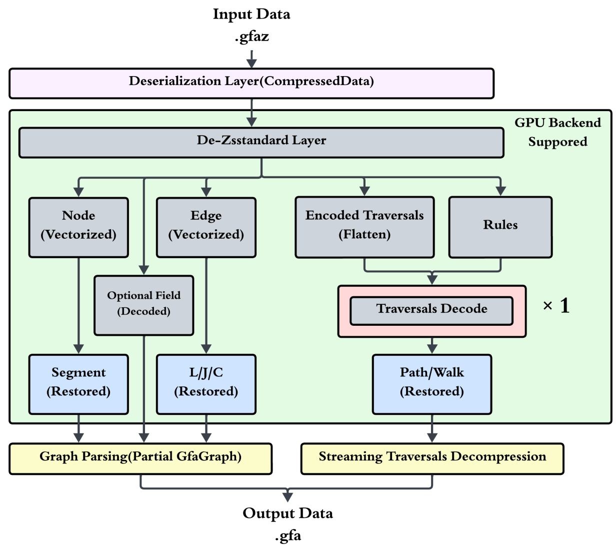


Figure 3: GFAz decompression workflow. Traversal expansion runs in linear time and is dependency-free across records, enabling batched parallel decode and direct streaming output without rebuilding the full graph in memory.

3.4 Decompression Pipeline

Decompression is partly symmetric to compression. For segments, links, jumps, containments, and optional fields, the decoder simply applies the inverse of the field-specific encodings used in Phase 3 and Phase 4: ZSTD decompression, inverse delta, varint decoding, and orientation bit-unpacking. These field restorations are independent and run in parallel sections.

Traversal decoding is intentionally asymmetric. Rather than rebuilding a full in-memory `GfaGraph`, GFAz expands P/W records with a streaming pipeline. The decoder directly uses `rules_first`, `rules_second`, and `start_id`; no extra rulebook reconstruction step is required. Traversals are partitioned into batches, multiple threads expand and decode one batch at a time, and each batch is written directly as GFA lines to the output file. This design keeps peak memory bounded by the working batch rather than by the total size of all paths and walks.

For each encoded traversal element e:

(1) **Check**: If $|e| <$ start_id, it is a literal node ID; we append it directly to the output.
(2) **Expand**: If $|e| \geq$ start_id, it is a rule. We resolve its children via direct array lookup at index ($|e| -$ start_id).
(3) **Recurse**: This lookup mimics a depth-first traversal of the grammar tree. We recursively expand the children until we reach literals. Crucially, if the rule ID e is negative, we expand the children in *reverse order* and flip their signs.

After expansion, inverse delta is applied D times to recover the original node IDs. The recovered oriented IDs are then converted immediately back into the textual P-line or W-line syntax and appended to the output stream. Because traversals are independent, this batched decode-and-write procedure parallelizes naturally across CPU threads. After traversal expansion, the rule table is freed to reclaim memory. This design ensures high throughput. Rule resolution is strictly $O(1)$ (an array read), involving no hash table lookups or complex pointer chasing.

Although the CPU and GPU backends may produce different rule sets and encoded traversal streams, both serialize into the same format (`rulebook + start_id + encoded traversals`), so either backend can correctly decompress files produced by the other.

3.5 Incremental Haplotype Extension and Random Access

GFAz supports random access to individual haplotypes and incremental extension by appending new haplotypes. Both operations decode only the lightweight traversal vectors, metadata columns, and stored grammar rules, without reprocessing the full graph.

Random Access. The `extract-traversal` operation reconstructs only the requested haplotype. GFAz locates the target path or walk from its metadata columns, slices the corresponding encoded subsequence from the flattened traversal vector, expands the grammar rules, applies inverse delta decoding, and emits only that record.

Incremental Extension. The `add-haplotypes` operation appends new paths or walks using the rulebook already stored in the input `.gfaz` file. GFAz validates the new haplotypes, applies the delta transform, encodes only the appended paths or walks, and writes back the updated columns. To preserve decodability, the appended traversal values must remain outside the rule-ID range. Let `start_id` denote the first rule ID stored in the existing file. GFAz verifies that the maximum delta value in the appended traversals is smaller than `start_id`, ensuring that these literals cannot be misinterpreted as grammar rules during later decoding.

3.6 GPU Backend

GFAz implements a high-performance GPU backend designed to keep the *traversal compression loop* device-resident. After transferring the traversal stream to GPU memory, all per-round operations—2-mer mining, rule table construction, parallel rule application, and stream compaction—execute on the GPU, avoiding the CPU-side dependency chain that constrains the serial greedy algorithm.

Parallel 2-mer Mining via Sorting. The most computationally intensive step is identifying repeated 2-mers. Traditional hash-counting is unsuitable for the GPU due to atomic contention. Instead, we implement a dependency-free, sort-based pipeline given a vector of node IDs D of length N:

(1) **GenerateKeys (Canonical 2-mer Keys)**: Since the input size N is known, we allocate a device vector of size $M = N - 1$. A massively parallel kernel launches one thread per index $i \in [0, M)$ and writes a canonical packed 2-mer:

$$K_i = \min(\text{pack}(D_i, D_{i+1}), \text{pack}(-D_{i+1}, -D_i)).$$

This step is embarrassingly parallel and requires no synchronization.
(2) **SortKeys (Group Identicals)**: We sort K on the GPU (radix sort) so that identical 2-mers form contiguous runs.
(3) **MarkDuplicates (Run Membership)**: In parallel, we compute a boolean flag F_i that marks whether K_i belongs to a run,

Algorithm 2 GPU Parallel 2-mer Mining

Input: Device vector D (int32 nodes), integer N
Output: Device vector *Repeats* (unique int64 2-mers)
$M \leftarrow N - 1$
$Keys \leftarrow$ **ALLOCATE_DEVICE_VECTOR**(M)
{1. GenerateKeys: canonical packed 2-mers}
PARALLEL_FOR $i \in \{0 \ldots M-1\}$
  $K_f \leftarrow \textsc{Pack}(D_i, D_{i+1})$
  $K_r \leftarrow \textsc{Pack}(-D_{i+1}, -D_i)$
  $Keys_i \leftarrow \min(K_f, K_r)$ {canonicalization}
END_PARALLEL
{2. SortKeys: group identical 2-mers}
SortKeys($Keys$) {radix sort}
{3. MarkDuplicates: flag run membership}
$Flags \leftarrow$ **ALLOCATE_DEVICE_VECTOR**(M) {uint8}
PARALLEL_FOR $i \in \{0 \ldots M-1\}$
  $left \leftarrow (i > 0) \wedge (Keys_i = Keys_{i-1})$
  $right \leftarrow (i+1 < M) \wedge (Keys_i = Keys_{i+1})$
  $Flags_i \leftarrow (left \vee right)$
END_PARALLEL
{4. CompactUnique: extract repeats and uniquify}
$RawRepeats \leftarrow$ **COMPACT**($Keys$, $Flags == 1$) {copy_if}
$Repeats \leftarrow$ **UNIQUE**($RawRepeats$)
return *Repeats*

Algorithm 3 GPU Parallel Checkerboard Encoding

Input: Device vector D (int32), device hash table T
Output: New device vector D', per-round `rules_used`
Allocate arrays F (uint8, size N) and V (int32, size N)
{F_i: 0 literal, 1 selected start, 2 consumed}
{1. Candidate marking (no dependencies)}
MarkCandidates(D, T) $\longrightarrow (F, V)$
{$F_i \in \{0, 1\}$ after this step}
{2. Checkerboard selection (two kernel-wide phases)}
SelectEvens(F) {for even i with F_i=1: set F_{i+1}=2}
ResolveOdds(F) {for odd i with F_i=1: cancel if F_{i+1}=1, else set F_{i+1}=2}
{3. Emit + compact}
$D' \leftarrow$ ScatterCompact(D, V, F) {emit V_i if F_i=1, else D_i; skip F_i=2}
MarkUsedRules(F, V) $\longrightarrow$ `rules_used`

using only neighbor comparisons:

$$F_i = \begin{cases} (K_0 = K_1), & i = 0, \\ (K_i = K_{i-1}) \ \vee \ (K_i = K_{i+1}), & 0 < i < M-1, \\ (K_{M-1} = K_{M-2}), & i = M-1. \end{cases}$$

(4) **CompactUnique (Extract Repeats)**: We apply stream compaction (`copy_if`) to extract all keys with $F_i = 1$, then apply `unique` to produce the final set of repeated 2-mers (frequency ≥ 2) used as candidate rules for this round.

Parallel Checkerboard Encoding. In the CPU backend, rule substitution is a serial, greedy scan: a replacement at index i consumes $(i, i+1)$ and therefore blocks a replacement starting at $i+1$. This induces a strict left-to-right dependency chain. On the GPU, we remove this dependency with a two-phase "checkerboard" schedule that selects a non-overlapping set of rule starts using only local neighbor state and two global phases (even pass, then odd pass):

(1) **MarkCandidates (Lookup + Orientation)**: For each $i \in [0, N-2]$, one thread forms the canonical key K_i and performs a hash-table lookup in T. If found, let $r_i = T[K_i]$ be the rule ID assigned to this 2-mer key; we mark i as a candidate start and store the signed replacement value: $+r_i$ if pack$(D_i, D_{i+1}) = K_i$, else $-r_i$.
(2) **SelectEvens**: All even candidates $i \in \{0, 2, 4, \ldots\}$ are selected simultaneously. Because intervals $[2k, 2k+1]$ do not overlap, we can safely mark each right neighbor i+1 as *consumed.*
(3) **ResolveOdds**: Odd candidate starts $i \in \{1, 3, 5, \ldots\}$ are selected only if their right neighbor i+1 is *not* an even start. If i+1 is an even start, the odd candidate is canceled; otherwise it is selected and consumes i+1.
(4) **ScatterCompact**: After selection, F_i encodes three states: literal (0), selected start (1), or consumed (2). We keep indices not marked consumed, emitting the signed rule ID at selected starts and the original literal otherwise. A `rules_used` bitmap is built in the same pass.

Compact, Sort, and Remap. Following encoding, we perform the same optimization steps as the CPU backend but strictly using GPU primitives. We use a parallel prefix sum (scan) to calculate the destination indices for active rules (compaction). The rules are then sorted by their 2-mer keys using GPU Radix Sort to ensure optimal delta encoding. Finally, a parallel kernel remaps all rule occurrences in the traversal vector to their new sorted IDs. The final output matches the CPU pipeline exactly: a single flattened `int32` traversal vector, two parallel rule definition vectors, and `start_id`, allowing for identical downstream processing.

Field Encoding. Once traversals and rule tables are produced as contiguous device vectors, we compress them directly on the GPU using `nvComp` ZSTD. For boolean columns (e.g., orientations), we first bit-pack flags (1 bit/value) and then apply ZSTD; for integer and byte streams we apply ZSTD directly, leveraging the structure created by sorting, delta encoding, and compaction.

GPU Decompression (Traversals). A direct GPU port of CPU-style recursive expansion (depth-first rule chasing) performs poorly: varying grammar depth introduces divergence, and repeated rule lookups lead to irregular memory access. We therefore use a *rule pre-expansion* strategy that converts recursive decoding into a small number of dense, regular memory passes:

(1) **PreExpandRules (Once per Rulebook)**: We pre-expand every rule into a contiguous buffer. Concretely, we compute each rule's final expansion length (number of literals after full expansion), exclusive-scan these lengths to obtain per-rule offsets, then materialize a dense array that stores each rule's fully expanded literal sequence. Negative rule references are handled by reversing child order and flipping signs during expansion.
(2) **ComputeOffsets (Per Traversal)**: For each element of the encoded traversal, we compute its output length (1 for raw node IDs; `rule_sizes[|r| − min_rule_id]` for rules) and perform an exclusive scan to obtain the exact output write position for every element.

Table 4: Evaluation datasets.

Name	Type	Size
`chr1.pan.smooth.gfa`	Human chr (smooth)	7.1 GB
`chr6.pan.smooth.gfa`	Human chr (smooth)	3.8 GB
`EcoliGraph_MGC.gfa`	Microbial (*E. coli*)	0.113 GB
`hprc-v1.1-mc-chm13.gfa`	Human WGS (HPRC)	48 GB
`hprc-v2.0-mc-chm13.gfa`	Human WGS (HPRC)	358 GB
`hprc-v2.1-mc-chm13.gfa`	Human WGS (HPRC)	369 GB

(3) **ExpandTraversalSinglePass (Scatter/Copy)**: A single kernel writes the decoded traversal into the output buffer: literals are copied directly; rules are expanded by bulk-copying their pre-expanded segments from the dense rule buffer. If a rule ID is negative, we copy the segment sequence in reverse order and negate each element.

Compared to an alternative reverse-n-round decoder that undoes grammar layers one level at a time (which requires repeated full-vector *scan* → *expand* → *re-scan* passes and global synchronization), this pre-expansion strategy removes the bottleneck of multiple kernel launches. Variable-depth recursion is paid once per rulebook, and traversal decoding becomes a small constant number of regular passes. This increases throughput from about 30 GB/s (using our deprecated iterative expander) to over 300 GB/s. In practice, the decode phase is bandwidth-dominated, followed by GPU-resident inverse delta (prefix sum) to recover original node IDs. Other fields are decoded and copied back to host memory.

4 Evaluation

4.1 Experimental Setup

All experiments run on a workstation with an AMD Ryzen Threadripper PRO 9955WX (16 cores), an NVIDIA RTX Pro 6000 GPU, and 512 GB of DDR5 6400 MHz memory. For CPU compressors that support multithreading, we enable all 16 hardware threads. Unless otherwise noted, `zstd` uses compression level 9. To measure algorithmic throughput rather than storage performance, we report *warm-cache* timings averaged over 5 runs: before each run, we prime the OS page cache by streaming the input file once (e.g., `cat input.gfa > /dev/null`) to minimize disk I/O effects.

4.2 Dataset

We evaluate our compressor on the datasets summarized in Table 4. For readability, we refer to the chromosome-scale smoothed graphs using short names (`chr1.pan.smooth.gfa` and `chr6.pan.smooth.gfa`) [11]. These datasets are representative for three reasons. The chromosome scale PGGB graphs (chr1/chr6) provide realistic human pangenome structure at a manageable scale, making it easier to stress specific components (e.g., traversal redundancy) without the full whole-genome footprint. Second, the HPRC minigraph–cactus graphs span multiple rapidly evolving public releases (v1.1, v2.0, v2.1) [11], allowing us to evaluate robustness as graph construction pipelines and graph complexity change over time. Finally, `EcoliGraph_MGC.gfa` [16] tests generality beyond human genomes on a microbial pangenome with dense path sets and different repeat/variation characteristics.

Table 5: Compression ratio, compression (Co.) and decompression (De.) throughput (MB/s) across datasets. Blue indicates the best performance, green denotes the second best.

Dataset	metrics	Gzip	Zstd	sqz	sqz+bgzip	sqz+Zstd	GBZ	GFAz (CPU)	GFAz (GPU)
chr1.	Ratio	5.59	7.54	3.09	18.0	16.7	9.52	35.4	31.7
	Co.	46.2	2178	3.95	3.97	4.00	12.1	385	2754
	De.	359	1618	21.6	21.4	21.2	284	1658	8124
chr6.	Ratio	5.04	6.99	5.51	20.8	19.2	19.2	35.4	28.18
	Co.	41.0	1712	3.56	3.56	3.59	10.7	348	3791
	De.	348	1515	20.1	20.4	20.2	281	1758	7230
E.coli	Ratio	4.69	5.67	1.26	7.46	6.79	5.58	18.3	16.7
	Co.	33.3	1356	4.57	4.53	4.62	20.2	190	678
	De.	310	1258	34.0	32.2	34.5	197	491	1430
HPRCv1.1	Ratio	4.02	5.32	-	-	-	14.0	22.4	20.4
	Co.	36.4	1657	-	-	-	84.5	231	4843
	De.	319	1234	-	-	-	650	1058	9435
HPRCv2.0	Ratio	4.19	6.49	-	-	-	66.8	83.9	76.4
	Co.	49.1	1514	-	-	-	130	367	-
	De.	342	1240	-	-	-	648	1652	-
HPRCv2.1	Ratio	4.19	6.43	-	-	-	64.2	82.8	74.2
	Co.	48.9	1540	-	-	-	136	348	-
	De.	343	1241	-	-	-	652	1559	-

4.3 Compression-Decompression Performance

We include gzip and zstd as general-purpose baselines, GBZ and SQZ as pangenome-oriented compressors; for SQZ we also report `sqz+bgzip` and `sqz+zstd` to probe its best ratio under alternative back-end compressors. SQZ runs out of memory on HPRC v1.1 and larger graphs, so those entries are marked NA. SQZ and GBZ also drop parts of the original GFA representation, making the GFAz comparisons conservative. For the 300+ GB HPRC v2.0/v2.1 graphs, the current GPU implementation runs out of device memory; we therefore report GPU ratios via a CPU simulation of the GPU rule-mining/encoding logic and omit GPU throughput for those two datasets. Table 5 reveals three consistent patterns.

Insight 1 (ratio gains increase with graph complexity). GFAz gives the best ratio on all datasets with available results, for both backends. On chr1, GFAz (CPU) reaches 35.4× versus zstd 7.54× and GBZ 9.52×; on chr6, 35.4× versus 6.99× and 19.2×. On larger HPRC graphs, the gap widens: on v2.0, GFAz (CPU) reaches 83.9× versus zstd 6.49× and GBZ 66.8×; on v2.1, 82.8× versus 6.43× and 64.2×. The GPU ratio is consistently but modestly below CPU (e.g., 31.7× vs 35.4× on chr1, 20.4× vs 22.4× on v1.1), consistent with checkerboard encoding (Section 3.6) replacing fewer overlapping

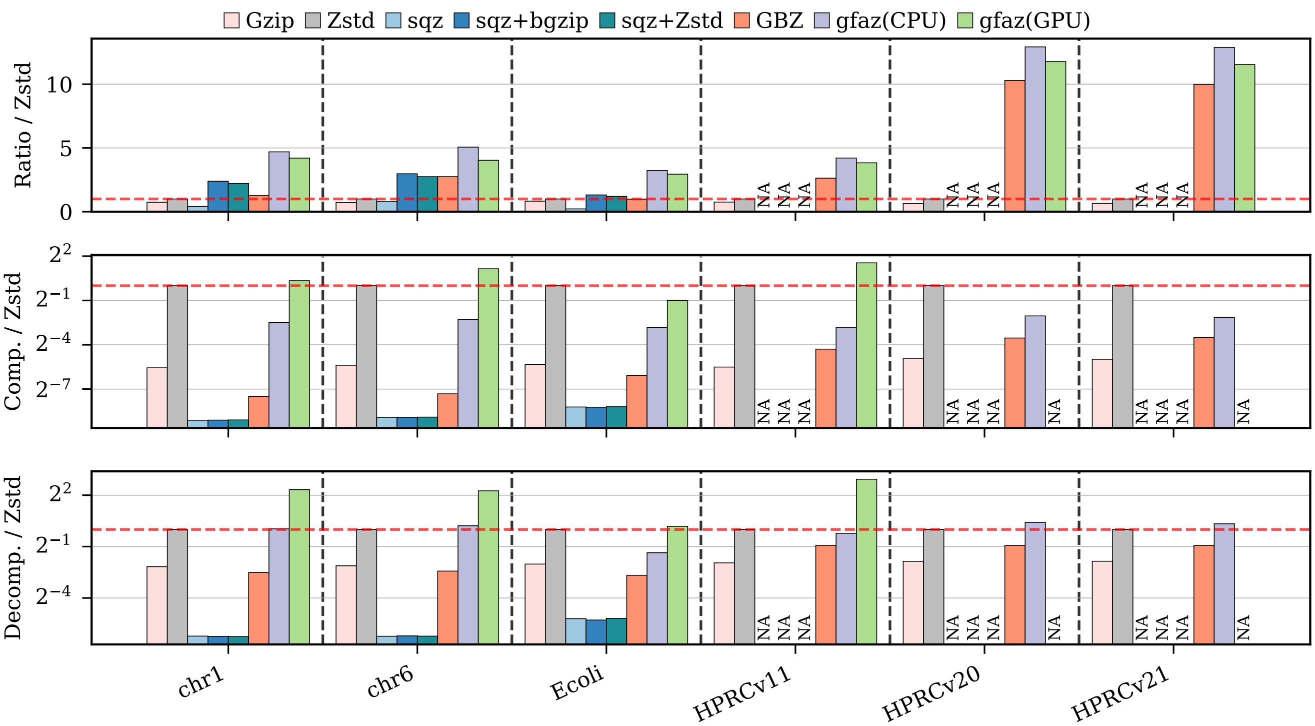


Figure 4: Normalized compression ratio, compression speed, and decompression speed relative to Zstd across all datasets. Results are normalized such that Zstd = 1.0 (dashed horizontal line). Speed metrics are plotted on a log scale (base 2).

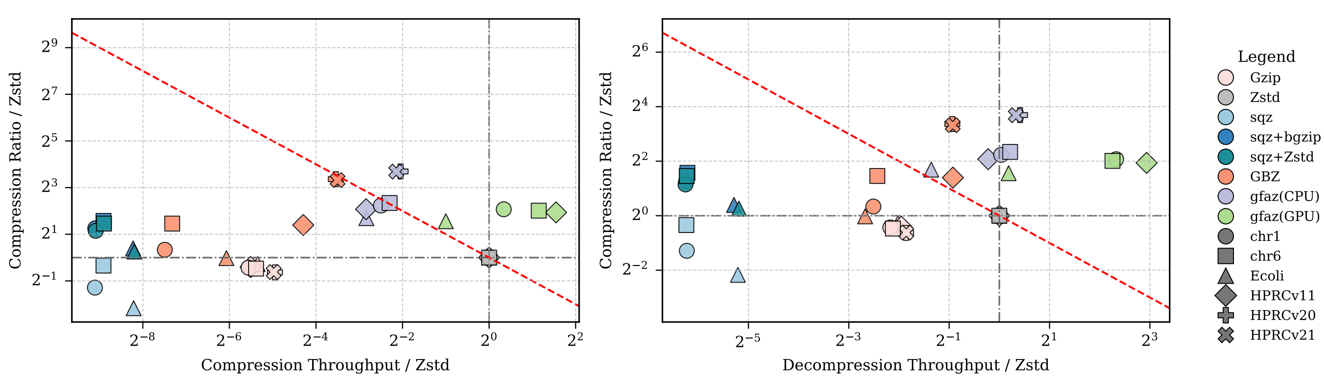


Figure 5: Pareto trade-off between compression ratio (y-axis) and speed (x-axis), normalized to Zstd (baseline at 1.0). The dashed diagonal line ($y = 1/x$) represents an iso-efficiency curve where a 2× increase in ratio compensates for a 2× drop in speed. Points above this line and to the right represent better trade-offs than the baseline.

opportunities than serial greedy encoding in each round. The ratio advantage is not specific to the HPRC pipeline. Table 7 adds two graphs from other sources: HGSVC3, an 80.1 GB whole-genome Minigraph–Cactus pangenome from a different consortium, and a chromosome from the 1000-genome-scale 1KCP collection whose variation is carried entirely in graph topology (a reference-only GFA with no embedded haplotype paths). On both, GFAz (CPU) again gives by far the best ratio—28.1× on HGSVC3 and 18.4× on 1KCP-chr1, versus 4.0–4.5× for gzip and 6.9–7.1× for zstd.

Insight 2 (Throughput remains strong across CPU/GPU). When inputs fit GPU memory, GFAz (GPU) is the fastest high-ratio operating point: on HPRC v1.1, it reaches 4843/9435 MB/s compression/decompression, versus GFAz (CPU) at 231/1058 MB/s and zstd at 1657/1234 MB/s. On chr1 and chr6, GFAz (GPU) also leads throughput, showing that the GPU backend is most effective

Table 6: Ablation of `Compact` and `Sort` on rulebook effectiveness. Each cell shows H / L, where $H = r \gg 32$ and $L = r \,\&\,$ `0xffffffff`; each entry is the ZSTD-compressed size followed by the compression ratio in parentheses.

Dataset	compact+sort	compact	sort	none
EcoliGraph_MGC	53.9 KB (**9.6×**) / 215 KB (**2.4×**)	348 KB (1.5×) / 358 KB (1.4×)	70.7 KB (9.6×) / 291 KB (2.3×)	463 KB (1.5×) / 477 KB (1.4×)
chr1.pan.smooth	4.3 MB (**11.2×**) / 30.4 MB (**1.6×**)	40.6 MB (1.2×) / 40.6 MB (1.2×)	5.2 MB (11.1×) / 38.0 MB (1.5×)	50.0 MB (1.2×) / 50.0 MB (1.2×)
chr6.pan.smooth	2.0 MB (**12.1×**) / 14.0 MB (**1.7×**)	19.3 MB (1.2×) / 19.3 MB (1.2×)	2.4 MB (12.0×) / 17.8 MB (1.6×)	24.1 MB (1.2×) / 24.1 MB (1.2×)
hprc-v1.1	61.2 MB (**10.2×**) / 330 MB (**1.9×**)	565 MB (1.1×) / 561 MB (1.1×)	73.9 MB (10.4×) / 418 MB (1.8×)	697 MB (1.1×) / 692 MB (1.1×)
hprc-v2.0	124 MB (**13.1×**) / 1254 MB (**1.3×**)	1512 MB (1.1×) / 1513 MB (1.1×)	153 MB (13.0×) / 1558 MB (1.3×)	1857 MB (1.1×) / 1859 MB (1.1×)
hprc-v2.1	130 MB (**13.1×**) / 1315 MB (**1.3×**)	1588 MB (1.1×) / 1589 MB (1.1×)	161 MB (12.9×) / 1632 MB (1.3×)	1949 MB (1.1×) / 1951 MB (1.1×)

Table 7: Compression ratio on two graphs from outside the HPRC (single-run, ratio only). GFAz retains its ratio advantage on a different-consortium pangenome (HGSVC3) and on a topology-only graph (1KCP-chr1).

Dataset	GFA	Gzip	Zstd	GFAz (CPU)
HGSVC3	80.1 GB	4.0×	7.1×	**28.1×**
1KCP-chr1	1.66 GB	4.5×	6.9×	**18.4×**

Table 8: Random-access evaluation using `extract-traversal`. We measure the time and peak memory required to reconstruct only 1, 4, or 16 selected paths/walks from an existing `.gfaz` file.

Dataset	Extracted	Elapsed (s)	Peak Mem. (GB)	GFA Size (GB)
chr1	1	0.102	0.38	7.1
	4	0.103	0.37	7.1
	16	0.106	0.41	7.1
hprc-v1.1	1	1.006	4.09	48
	4	1.012	4.10	48
	16	1.029	4.13	48
hprc-v2.0	1	2.285	9.26	358
	4	2.302	9.28	358
	16	2.312	9.30	358
hprc-v2.1	1	2.278	9.65	369
	4	2.303	9.67	369
	16	2.380	9.74	369

when traversal data can remain device-resident. For v2.0/v2.1, GPU throughput is unavailable because the traversal stream exceeds device memory, but GFAz (CPU) still gives the fastest decompression (1652 and 1559 MB/s) while retaining much higher ratios than zstd. CPU compression is slower than zstd on these largest graphs, but this trade-off is favorable for storage and repeated-read workloads. On the small *E. coli* input, zstd compresses fastest, while GFAz (GPU) still improves both ratio and decompression speed.

Insight 3 (alternative methods occupy weaker ratio–speed fronts). SQZ variants can improve ratio over gzip/zstd on chromosome graphs, but their throughput (about 3–5 MB/s compression and about 20–35 MB/s decompression) is two orders of magnitude lower than GFAz. GBZ is the strongest non-GFAz ratio baseline on large HPRC graphs, but remains below GFAz in ratio (66.8×/64.2× vs 83.9×/82.8× on v2.0/v2.1) and decompression speed (648/652 vs 1652/1559 MB/s on CPU), while preserving fewer GFA semantics.

Normalized and Pareto views. Figures 4 and 5 summarize the same results normalized to Zstd. GFAz consistently improves decompression and ratio efficiency. For compression, only the small

Table 9: Incremental haplotype extension using `add-haplotypes`. Each run appends 50 new paths/walks using the rulebook already stored in the input `.gfaz` file.

Dataset	Added Haplotypes	Elapsed (s)	Peak Memory (GB)	GFA Size (GB)
chr1	50	0.95	0.97	7.1
chr6	50	0.54	0.51	3.8
hprc-v1.1	50	3.25	3.25	48
hprc-v2.0	50	9.80	16.85	358
hprc-v2.1	50	10.20	17.32	369

input can fall below the line but still keeps a substantial multi-fold ratio advantage over Zstd.

4.4 Random Access and Incremental Extension

We next evaluate the two traversal-centric operations enabled by the `.gfaz` layout: extracting a small number of haplotypes without full decompression, and appending new haplotypes without recompressing the entire file.

Random access. Table 8 reports extraction of 1, 4, or 16 paths/walks from existing `.gfaz` files. Runtime stays nearly flat as more records are requested. On HPRC v2.1 (369 GB), extracting 1, 4, and 16 haplotypes takes 2.28 s, 2.30 s, and 2.38 s, respectively. Peak memory also changes only slightly, from 9.65 GB to 9.74 GB.

Incremental extension. Table 9 reports the cost of appending 50 new haplotypes. This takes 0.54–0.95 s on chromosome-scale graphs, 3.25 s on HPRC v1.1, and about 10 s on the 358–369 GB graphs. On HPRC v2.1, appending 50 haplotypes takes 10.2 s, compared with roughly 1100 s for full recompression.

4.5 Ablation: Compact and Sort

We evaluate four workflow variants: `compact+sort` (full design), `compact` only, `sort` only, and `none`. For each packed 64-bit rule ID r, we split it into two 32-bit integer streams: $H = r \gg 32$ (high 32 bits) and $L = r \,\&\,$ `0xffffffff` (low 32 bits) during compression, then entropy-code each stream with ZSTD. Results directly validate the design rationale in Section 3.3. Sorting is the dominant factor for H: on chr1, H improves from 1.2× (`none`) to 11.1× (`sort`), and to 11.2× with `compact+sort`; similarly on hprc-v2.1, 1.1× (`none`) to 12.9× (`sort`) and 13.1× (`compact+sort`). Compact then provides additional gains by removing unused rules and shrinking emitted streams: for chr1 L improves from 1.5× (`sort`) to 1.6× (`compact+sort`), and for hprc-v2.1 from 1.3× to 1.3× with a concrete size drop (1632 MB to 1315 MB). Compared with `compact` only, the full `compact+sort` pipeline is consistently much better on H (e.g., chr6: 12.1× vs 1.2×), showing that compacting without ordering does not create enough entropy structure for the rulebook. This

Table 10: CPU stage-level profiling for compression and decompression. Share is computed as `time / stage total`.

Stage	Time (ms)	Tput (GB/s)	Share
Compression (total: 19888.78 ms)			
Grammar subtotal	11004.68	0.20	55.3%
generate_rules	6000.89	0.38	30.2%
encode_traversals	4785.13	0.47	24.1%
compact_sort_remap	196.83	11.5	1.0%
split+delta_encode	27.93	3.30	0.1%
zstd compress	467.58	0.47	2.4%
Others	8884.10	–	44.7%
Decompression (total: 4221.47 ms)			
Grammar subtotal	511.52	4.32	12.1%
zstd decompress	102.39	1.60	2.4%
decode rules (prefix sum)	2.85	32.4	0.1%
expand traversals	371.93	6.06	8.8%
decode traversals (pfx sum)	34.35	65.6	0.8%
Others	3709.95	–	87.9%

Table 11: GPU stage-level profiling for compression and decompression. Share is computed as `time / stage total`.

Stage	Time (ms)	Tput (GB/s)	Share
Compression (total: 2322.85 ms)			
Grammar subtotal	571.85	3.96	24.6%
find_max_abs	5.96	378.0	0.3%
delta_encode	5.64	400.0	0.2%
find_repeated_2mers	195.59	11.5	8.4%
create_rule_table	18.36	122.8	0.8%
apply_2mer_rules	105.24	21.4	4.5%
compact_rules_remap	25.68	87.8	1.1%
sort_rules_remap	16.70	135.0	0.7%
split+delta_encode	0.90	102.2	0.0%
Zstd compress (nvComp)	197.78	11.4	8.5%
Others	1751.00	–	75.4%
Decompression (total: 876.00 ms)			
Grammar subtotal	117.00	19.3	13.4%
Zstd decompress (nvComp)	85.85	26.3	9.8%
decode rules (prefix sum)	0.44	367.2	0.1%
expand traversals	24.73	91.2	2.8%
decode traversals (pfx sum)	5.98	377.1	0.7%
Others	759.00	–	86.6%

also clarifies a weakness of SQZ: although SQZ is grammar-based, highly variable rule symbols without strong ordering reduce rule-stream regularity, making entropy coding less effective than the sorted rulebook layout used by GFAz.

4.6 Performance Breakdown

Tables 11 and 10 break down stage-level profiling for the GPU and CPU backends respectively. Each table reports a `Grammar subtotal` (traversal-related stages) and `Others` (all remaining fields—segment sequences, link/jump/containment columns, optional fields, and their associated ZSTD encoding).

On both backends, decompression is dominated by `Others`: 87.9% on CPU and 86.6% on GPU, leaving only 12.1% (CPU) and 13.4% (GPU) in grammar stages. Within grammar decode, traversal expansion itself is fast (371.93 ms at 6.06 GB/s on CPU; 24.73 ms at 91.2 GB/s on GPU), consistent with one-pass expansion and $O(1)$ rule lookup (`rule_id - min_rule_id`). This supports the design claim that traversal reconstruction is not the end-to-end bottleneck once the rulebook is available.

The two backends differ mainly in compression cost distribution. On CPU, grammar stages consume 55.3% of compression time, with `generate_rules` (30.2%) and `encode_traversals` (24.1%) as dominant kernels. On GPU, grammar drops to 24.6% of total compression time (`find_repeated_2mers` 8.4%, `apply_2mer_rules` 4.5%), while `Others` rises to 75.4%, consistent with nvComp/ZSTD orchestration overhead in the current pipeline. This profile explains why the GPU grammar algorithm is already fast but end-to-end speedups are now limited by entropy and non-traversal stages.

Table 12: CPU peak resident memory using 16 threads for compression, a legacy decompression baseline that fully materializes the in-memory `GfaGraph`, and the implemented streaming decompression path.

Dataset	Comp. Peak (GB)	Full-Graph Decomp. (GB)	Streaming Decomp. (GB)	GFA Size (GB)
chr1	7.4	6.3	0.98	7.1
chr6	3.2	3.0	0.66	3.8
Ecoli	0.24	0.18	0.18	0.113
hprc-v1.1	45	37	3.4	48
hprc-v2.0	343	289	19.3	358
hprc-v2.1	356	295	20.7	369

4.7 CPU Memory Profiling

Table 12 compares three CPU memory profiles measured with 16 threads: compression, a legacy decompression baseline that fully materializes the entire `GfaGraph`, and the implemented streaming decompression path that expands one path/walk and writes it out immediately. The fully materialized baseline reaches 37 GB, 289 GB, and 295 GB on HPRC v1.1, v2.0, and v2.1, respectively. In contrast, the implemented streaming path reduces peak decompression memory to 3.4 GB, 19.3 GB, and 20.7 GB, a reduction of about 14–15×.

High compression peak memory is less critical in practice for three reasons. First, it occurs primarily in the first rule-encoding round and drops quickly afterward as the traversal stream shrinks. Second, compression is usually a one-time preprocessing step, whereas decompression and downstream analyses are run repeatedly. Third, downstream graph tools such as VG [5] and ODGI [7] already require memory footprints larger than the input GFA size.

5 GFAz as a Compute Engine

The previous sections present GFAz as a compressor. We now show that the same container is also a *compute substrate*: path-centric pangenome analyses run directly on the grammar-compressed traversal store, without ever reconstructing the GFA text or building a whole-graph object. Two observations make this possible. The dominant component of a pangenome graph—the haplotype traversals, over 90% of the file—is already stored in a streamable, grammar-compressed form. And most downstream analyses do not mutate the graph: they scan traversals and fold node visits into a compact accumulator, such as a coverage histogram, a window-by-sample matrix, or an allele table relative to a reference path.

5.1 Overview and Design Contributions

The compute engine contributes along two axes.

- **Computing over compression.** A streaming execution model evaluates every analysis directly on the compressed traversal store. The traversal columns are never expanded into a materialized list of path and walk steps; a query pays only for the grammar-compressed columns it must touch plus a small accumulator. The consequence is order-of-magnitude speedups over graph- or text-resident tools and a peak memory footprint that stays *below the original uncompressed GFA*, even on whole-genome human pangenomes.
- **`deconstruct`: a new algorithm for GFA→VCF projection.** The flagship analysis derives a reference-relative VCF—the entry point of the downstream genomics toolchain—directly from compressed traversals. It is a new two-phase algorithm that separates a one-time, topology-only site-discovery pass from a path-centric allele-observation fold, so it never materializes the graph or a general snarl index that graph-resident deconstruction relies on.

We first explain why deriving downstream products from a pangenome is a recurring cost and why current tools pay it through materialization (Section 5.2). We then develop the shared substrate—the streaming primitive (Section 5.3), the memory model and rule-leaf cache (Section 5.4), and the parallel execution strategy (Section 5.5). On that substrate we develop `deconstruct` in depth (Section 5.6), then five further analyses built the same way (Section 5.7).

5.2 Why Analyze on the Compressed Container

GFA is the format in which pangenome graphs are *built* and exchanged, but it is rarely the format in which they are *analyzed*. The established genomics toolchain—variant annotation, association testing, imputation, and clinical reporting—operates on tabular, reference-anchored products, not on graphs. The most important of these is the Variant Call Format (VCF): essentially the entire downstream ecosystem (`bcftools`, GATK, `plink`, and GWAS and imputation pipelines) consumes VCF, whereas a pangenome graph encodes variation only *implicitly*, as the alternative traversals that haplotypes take through its bubbles—a representation those tools cannot read. Other routine questions—how graph coverage is distributed, how the pangenome grows as haplotypes are added, how similar two assemblies are—are likewise answered today by exporting the graph into a separate, single-purpose tool for each product. Using a pangenome in practice therefore means repeatedly deriving these downstream artifacts from it.

Current tools pay for each derivation by first materializing a representation larger than the statistic they ultimately compute. Panacus [18] parses GFA path and walk records into item tables before deriving growth and core curves. odgi [7] builds a succinct whole-graph object before running operations such as `odgi pav`. `vg deconstruct` loads a graph and a snarl decomposition before emitting a reference-relative VCF from traversals [5, 9]. These representations are appropriate for general graph operations, but for path-centric folds the resident object dominates both memory and time. GFAz instead targets a narrower but ubiquitous execution pattern—compressed traversal stream → small accumulator → biological statistic—and pays only for the compressed traversal columns, the rulebook needed to expand them, and the state required by the specific query. It deliberately does not try to replace topology-heavy graph libraries; it exploits the fact that the dominant GFA component is already encoded in a streamable form.

5.3 The Compute Substrate

Orientation. Each step of a traversal carries an *orientation*. A segment can be read on either strand of its double-stranded DNA, and the sign of a node ID records which: a positive ID is the *forward* orientation, a negative ID the *reverse*. Reversing any contiguous run of a traversal is therefore a *reverse complement* —the steps are emitted in reverse order and every sign is flipped—just as the two strands of a sequence read in opposite directions. We refer to a segment's two ends as its 5′ and 3′ ends, and call a haplotype that crosses a site in the reverse orientation relative to the reference an *inversion*. These three notions —orientation, reverse complement, and inversion—recur throughout the engine and are central to `deconstruct`.

Compressed traversal store. The `.gfaz` container stores P traversals and W traversals as two flat `int32` arrays. Each record has an encoded-length column, and the original pre-compression traversal lengths are retained for workflows that need a materialized fallback. ZSTD decompression of these blocks yields grammar-compressed integer streams containing literals and rule IDs, but not the fully expanded node sequences. The engine slices the flat arrays into zero-copy `HapSlice` views:

$$\{\texttt{pointer},\ \texttt{encoded length},\ \texttt{original length}\}.$$

The grammar rulebook is stored as two parallel `int32` arrays, `rules_first` and `rules_second`, delta-decoded once after ZSTD decompression. The first rule ID, `min_rule_id`, is read from the layer metadata; `max_rule_id` is derived from the rule count. A value whose absolute ID lies in this half-open range is a rule, and all other values are literals. This single-comparison classification is sound because compression reserves the rule namespace strictly above every literal node ID (Section 3.3): rule IDs are assigned sequentially across rounds starting from `start_id`, so a decoder never has to consult a side table to tell a node from a rule.

Streaming primitive. The reusable operation is:

$$\textsc{StreamDecodedNodes}(\texttt{HapSlice},\ \textsc{Visit}).$$

It emits the decoded orientation-signed node IDs of one traversal, in order, to a caller-supplied visitor. For each encoded element e, the decoder performs one boundary check. If $|e|$ is outside the rule range, e is a literal. Otherwise the decoder indexes the rule arrays at $|e| - \texttt{min\_rule\_id}$ and expands the two children recursively. A negative rule reference denotes the *reverse complement* of the rule: its children are visited in reverse order with their orientations flipped. The same rulebook therefore serves both orientations without storing a reverse copy.

The inverse-delta transform is fused into streaming. For the common `delta_round=1` case, the decoder keeps a running prefix sum and calls the visitor immediately after reconstructing each node

Algorithm 4 Streaming traversal expansion for `delta_round` 0 or 1

Require: slice E, rule range $[m, M)$, rule arrays R_1, R_2, delta round $D \in \{0, 1\}$, visitor VISIT

$prev \leftarrow 0$
define EMIT(v):
if $D = 1$ **then**
    $prev \leftarrow prev + v$; VISIT($prev$)
else
    VISIT(v)
end if
define EXPAND(r, $reverse$):
    $(a, b) \leftarrow (R_1[r - m], R_2[r - m])$
    $(c_1, c_2) \leftarrow reverse\,?\,(b, a) : (a, b)$ {reverse swaps child order}
for all $c \in (c_1, c_2)$ **do**
    if $|c| \in [m, M)$ **then**
        EXPAND($|c|$, $reverse \oplus (c < 0)$) {rule child: fold parent and child orientation}
    else
        EMIT($reverse\,?\,-c : c$) {literal child: flip sign under reverse}
    end if
end for
for all $e \in E$ in order **do**
    if $|e| \in [m, M)$ **then**
        EXPAND($|e|$, $e < 0$)
    else
        EMIT(e)
    end if
end for

ID. For `delta_round=0`, literals and rule leaves are already in node-ID space. For uncommon higher delta counts, the implementation materializes one traversal, applies the required inverse passes, and then visits the recovered nodes.

The cost of decoding one traversal is linear in the expanded traversal length. Rule resolution is an array lookup, not a hash lookup, and recursion depth is bounded by the number of grammar rounds. The visitor is templated in the implementation, so the per-node fold can be inlined through the decode loop.

A shared extension surface. The streaming primitive is one element of a small traversal-query layer that every command reuses, so the engine is an extensible substrate rather than a fixed set of tools. The layer provides: rulebook and traversal loading that decompress the P/W blocks and expose them as zero-copy `HapSlice` views; the templated STREAMDECODEDNODES decoder above; the optional rule-leaf cache (Section 5.4); and a single PanSN grouping facility—one *grouping mode* (per record, sample, sample-haplotype, or sample-haplotype-sequence) with shared key functions for P- and W-lines—so every analysis aggregates haplotypes identically. A new analysis is then a short fold following one skeleton: deserialize the container, assign each P/W slice a group key, stream-decode slices into an analysis-specific visitor under a parallel loop, merge per-thread accumulators, and emit the statistic. Every command below is written this way and differs only in its accumulator and emission, not in how traversals are decoded.

5.4 Memory Model and the Rule-Leaf Cache

The compute engine replaces decompressed traversal materialization with at most four resident components:

(1) **Grammar-compressed traversal store**: ZSTD-decompressed but still grammar-encoded P/W integer streams, encoded-length columns, and the rule arrays.
(2) **Optional rule-leaf cache**: bounded by a configurable byte budget (below).
(3) **Analysis accumulator**: the state required by the specific query. Its shape follows the workload regime. A *path-iterative* fold keeps a compact accumulator independent of haplotype count—a per-node coverage or depth vector, a per-window denominator, or a per-site allele table—so its footprint is $O(\text{nodes})$ or $O(\text{windows})$. A *node-join* workload keeps a node-major index of which groups cover each node, in compressed-sparse-row (CSR) form, plus the output matrix it accumulates: $O(\text{windows}\times\text{groups})$ for `pav` and an upper-triangular $O(\text{groups}^2)$ for `similarity`.
(4) **Sequence payload when needed**: `deconstruct` additionally keeps segment DNA resident so it can spell REF and ALT alleles, and `stats` touches it only for an optional base-content tally.

The key distinction is that the traversal store remains grammar-compressed. Memory scales with the size of the compressed traversal columns and with the query accumulator, but not with a fully expanded list of all path and walk steps. Because the grammar-compressed columns are a fraction of the original GFA and the accumulators above are sublinear in the haplotype count, the peak footprint of every command stays below the size of the uncompressed input—the property that lets the engine analyze whole-genome pangenomes on commodity memory.

Rule-leaf cache. On-demand recursion avoids materializing all expanded traversals, but a large rule may be referenced many times. For workloads where repeated rule expansion is noticeable, GFAz provides a bounded `RuleLeafCache`: it stores the fully expanded forward leaf sequence for any rule whose expansion fits within a user-specified byte budget, and derives reverse expansion by scanning the cached leaves backward and negating signs, so no reverse copy is stored. Construction is bottom-up over rule IDs—a rule is admitted only if its children are already expanded or representable lazily and its projected leaf sequence fits the remaining budget—and rules that do not fit fall back to recursive expansion, preserving correctness for every budget. The cache is built once and shared read-only by all decoding threads. A budget of zero recovers fully lazy expansion; larger budgets trade memory for less repeated recursion. `pav` enables a non-trivial budget by default because its multi-pass structure revisits reference rules; the commands that stream every slice roughly once default the budget to zero and expose it through an environment variable for profiling.

5.5 Parallel Execution

Traversal decoding is parallel over `HapSlice` objects. The flat traversal arrays, rulebook, segment sequence payload, and rule-leaf cache are read-only after setup, so distinct threads can decode distinct traversals without synchronizing on the decode path. Because expanded traversal lengths vary widely across haplotypes, the parallel loops use dynamic OpenMP scheduling for load balance.

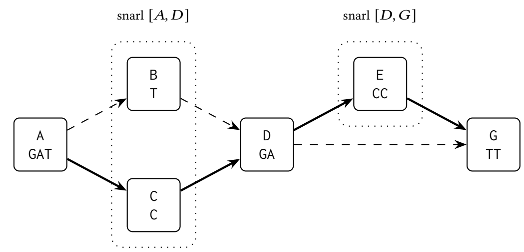
Synchronization is therefore confined to the workload-specific fold, never the decode itself, and the merge strategy follows the command's regime: a shared accumulator updated with fine-grained atomics, private per-thread state concatenated after the parallel region, or a parallel join over the node-major index. Because these policies are inseparable from each command's accumulator, we describe each one alongside its command in Sections 5.6–5.7 rather than in the abstract here. The compute engine is CPU-parallel throughout (OpenMP); the GPU backend (Section 3.6) accelerates compression and decompression, and a GPU port of the decode-and-fold loop is left to future work.

5.6 `deconstruct`: GFA→VCF as a Two-Phase Compressed-Traversal Algorithm

Deriving a reference-relative VCF is the most consequential downstream projection, and the most topology-sensitive analysis the engine supports: given a chosen reference path, `deconstruct` reports, for each site where haplotypes diverge from the reference, the distinct allelic spellings as a VCF record. The standard tool is `vg deconstruct` [5], which loads the whole graph, computes a snarl (superbubble) decomposition [19], and then, for each snarl, enumerates the traversals its embedded paths take and emits those differing from the reference as alleles. Both the materialized graph and the snarl index stay resident throughout, which makes the projection costly in time and memory at pangenome scale.

The two-phase idea. Our design departs from this in a single idea: *separate site discovery from allele observation.* Site discovery touches only the graph topology stored in the container and runs once in $O(V{+}E)$, decoding no haplotype. Allele observation is then the same compressed-traversal fold as every other engine command—each haplotype is streamed exactly once through compact site-boundary state. Neither phase materializes the graph or a general-purpose whole-graph snarl index, which is the source of the large speed and memory advantage over graph-resident pipelines. Before either phase, each reference contig is decoded once to an orientation-signed node stream `ref_nodes`; a prefix sum over segment lengths maps reference node positions to genomic offsets and gives the contig length used in the VCF header. Worked Example 1 traces one site through both phases.

Phase 1: topology-only site discovery. By default GFAz defines sites from top-level snarls, matching the record granularity of `vg deconstruct`, using a global biconnected-components (BCC) decomposition over the stored L-line topology. It builds an orientation-aware node-end graph: each segment s contributes two vertices, $s.5'$ and $s.3'$, joined by a *black* edge, and each GFA link contributes one *grey* edge between the source departure end and target arrival end. This is the bidirected "interval" graph of snarl theory, in which snarls correspond to biconnected blocks of the node-end representation [19]. The black segment edges are essential for inversions: a single-node inversion closes through the segment's two ends and is recovered as one biconnected block, whereas a collapsed segment graph would expose the inverted segment as an articulation point and split the site. GFAz runs an iterative Hopcroft–Tarjan decomposition over this graph (Algorithm 5), maintaining discovery times, low-link values, and an edge stack; when returning from child w to parent u, the condition $low(w) \geq disc(u)$ pops one maximal biconnected block. The pass is iterative rather than recursive to avoid call-stack limits on chromosome-scale graphs, and runs in $O(V+E)$ time with linear auxiliary space. Each non-trivial block is projected onto `ref_nodes`: if the first and last reference positions touched by the block define a clean span whose positions all belong to the block and contain no repeated segment, the span is emitted as a candidate top-level snarl. This filter rejects cyclic or ambiguous reference traversals, while nested child bubbles and chains of leaf bubbles collapse into their enclosing top-level snarl.



Reference path (and h_1): solid, allele 0. Haplotype h_2: dashed, allele 1.

```
#CHROM  POS  REF  ALT  QUAL  FILTER  INFO               FORMAT  h1  h2
chr1    4    C    T    .     .       AC=1;AN=2;AF=0.5   GT      0   1
chr1    6    ACC  A    .     .       AC=1;AN=2;AF=0.5   GT      0   1
```

Worked Example 1: `deconstruct` on two chained sites. Phase 1 recovers the snarls $[A, D]$ and $[D, G]$ from topology alone, decoding no haplotype; Phase 2 streams each haplotype once and reads each interior, taking the reference spelling as allele 0. The sites project to two VCF records—a substitution (`C`→`T`) and a left-anchored deletion (`ACC`→`A`).

Algorithm 5 Top-level snarl extraction via biconnected decomposition

Require: links L; reference node stream `ref`; segment count n

build node-end graph G: vertices $\{s.5', s.3'\}$ per segment s; *black* edge $(s.5', s.3')$; *grey* edge $(\textsc{dep}(a), \textsc{arr}(b))$ per link $a \to b$

$t \leftarrow 0$; edge stack $\Sigma \leftarrow [\,]$; $disc[\cdot] \leftarrow 0$

procedure $\textsc{Dfs}(u, pe)$: {entered via edge pe}

  $disc[u] \leftarrow low[u] \leftarrow (t \leftarrow t{+}1)$

  for each incident edge $e = (u, w) \neq pe$:

    if $disc[w] = 0$: push e on Σ; $\textsc{Dfs}(w, e)$; $low[u] \leftarrow \min(low[u], low[w])$

      if $low[w] \geq disc[u]$: pop Σ down to and including e as block B; $\textsc{Project}(B)$

    else if $disc[w] < disc[u]$: push e on Σ; $low[u] \leftarrow \min(low[u], disc[w])$

for each reference segment s and end $x \in \{s.5', s.3'\}$ with $disc[x] = 0$: $\textsc{Dfs}(x, \textsc{nil})$

procedure $\textsc{Project}(B)$:

  $S \leftarrow$ segments incident to B; $I \leftarrow \{\, i : \texttt{ref}[i] \in S \,\}$

  if $|B| \geq 2$ **and** $|I| \geq 2$: $(\ell, h) \leftarrow (\min I, \max I)$

    if $\texttt{ref}[\ell..h]$ repeats no segment **and** $\texttt{ref}[i] \in S\ \forall i \in [\ell, h]$: emit snarl $(\texttt{ref}[\ell], \texttt{ref}[h])$

reduce emitted snarls to a non-overlapping chain by position

Algorithm 6 Streaming allele observation for one haplotype (snarl mode)

Require: slice E; per-snarl boundary maps ENTRANCE (node-side $\rightarrow$ snarl, forward or reverse) and exit node-sides
open $\leftarrow \perp$; *interior* $\leftarrow$ [ ]
for each decoded node u of E (Algorithm 4):
  $v \leftarrow$ node-side vertex of u
if *open* $= \perp$ **then**
  TRYOPEN(v): if v is a (forward/reverse) entrance, set *open*, *inverted*, *exit*, *interior* $\leftarrow$ [ ]
else if v = *exit* **then**
  emit interior for snarl *open* (reverse-complement if *inverted*), tagged with this slice
  open $\leftarrow \perp$; TRYOPEN(v) {exit may open a chained snarl}
else
  append u to *interior*
end if

Phase 2: streaming allele observation. Once the sites are fixed, GFAz returns to the compressed traversal substrate. During streaming, a small state machine is either closed or associated with one open snarl (Algorithm 6). A boundary node-side opens a site, ordinary decoded nodes are appended to a small interior buffer, and the matching exit emits that interior tagged by the haplotype slice; a boundary may close one site and immediately open the next, which handles adjacent (chained) snarls without retaining the surrounding path. The boundary maps come in forward and reverse pairs so that a site crossed on either orientation is detected and normalized (see "Handling inversions" below). This observation runs in parallel over haplotypes, each thread appending its tagged interiors to a private buffer; the buffers are concatenated once the region ends, and the inherently serial per-site assembly and VCF formatting then run as a single pass, with each snarl's observations sorted by source slice so the emitted VCF is invariant to thread count.

Handling inversions. A haplotype need not cross a site in the reference orientation. When it traverses the interior on the opposite strand—an *inversion*—both the boundary node-sides and the interior nodes appear reverse-complemented, yet the engine must still recognize the site and recover the same allele. GFAz handles this at both phases. Phase 1 already does its part: the black segment edges above keep an inverted traversal within a single biconnected block, so the inversion is discovered as one site. Phase 2 then gives every snarl a second, reverse boundary pair: besides the forward entrance (reference source, exiting at the sink), GFAz stores a *reverse entrance* at the reverse complement of the sink, exiting at the reverse complement of the source. A haplotype that enters through the reverse entrance is marked inverted, and at its exit the interior is reverse-complemented—scanned backward with every orientation flipped—before the allele is spelled, normalizing the observation into the reference frame so that a forward and an inverted traversal of the same sequence collapse to one allele.

Assembly and VCF conventions. Per site, the reference interior spells allele 0; each observed interior is spelled by concatenating segment DNA (a reverse-orientation node contributes the reverse complement of its sequence) and deduplicated to an allele index. A site with no alternate allele is dropped. Equal-length alleles are emitted as substitutions; otherwise the record is left-anchored with the last base of the source anchor, following the standard VCF indel convention. `INFO` carries `AC`/`AN`/`AF` and the number of samples with data, and an optional `-max-site-length` guard collapses very large spans to a symbolic `<CPX>` record. Haplotypes collapse into VCF sample columns by the chosen grouping mode, with one genotype slot per haplotype index so reference-tiling contigs resolve to the sample's true ploidy rather than inflating it. Records are emitted in sorted genomic order per contig.

Validation. The two-phase algorithm is a new method, but its output is designed to be interchangeable with the established tool: on the HPRC chromosome graphs, GFAz's default mode agrees with `vg deconstruct` at about 99.99% of reference positions, producing the same sites and alternate-allele sets save for a handful of records in tangled pericentromeric regions. It does so without a materialized graph, which is the source of its time and memory advantage; we quantify both in the evaluation.

5.7 Further Analyses on the Same Substrate

Beyond `deconstruct`, GFAz implements five more analyses, each reproducing the output of an established tool while computing directly on the compressed container. They share the setup of Section 5.3—deserialize `CompressedData`, ZSTD-decompress the rulebook and P/W blocks, delta-decode the rule arrays, build `HapSlice` views, and reconstruct only the metadata columns needed for grouping or reference lookup—and split into two regimes. *Path-iterative* folds stream each haplotype once into a compact accumulator (`growth`, `depth`, and the metadata-only `stats`); *node-join* workloads build a node-major index of group coverage and join along shared nodes to emit a matrix (`pav` over reference windows, `similarity` over all group pairs). Of these, `pav` and `growth` carry design novelty of their own—a node-major join layout and a closed-form growth estimator—while `similarity`, `depth`, and `stats` demonstrate that the same substrate reproduces additional odgi outputs at a fraction of the cost. Table 13 summarizes all six.

PAV (node-join). This command computes a window-by-group matrix over BED intervals on reference paths or walks. Rather than reduce each haplotype to a scalar, it records *which groups cover each node* and joins along shared nodes—the shared node-join pattern, realized as a node-major compressed-sparse-row (CSR) index, chosen over a per-node container map to remove the per-node header overhead that is significant when the node count reaches hundreds of millions. It runs in three passes over a shared read-only decode state. *Pass 1* decodes every traversal once, parallel over slices and lock-free, into a private sorted-unique node set; reference traversals (those a BED `chrom` resolves to) additionally retain their ordered node stream, captured as a side effect of the same decode so no reference is decoded twice. *Pass 2* unions the per-slice node sets of each group, parallel over groups, and flattens the result into the node$\rightarrow$groups CSR in a single serial stitch: a counting pass tallies the groups per node, an exclusive prefix sum forms the row offsets, and a cursor scatter fills the group-ID column; the rule-leaf cache is released afterward, since Passes 2–3 perform no further grammar expansion. *Pass 3* sweeps each reference contig in parallel, maintaining a running genomic offset from the segment-length column so each visited node maps to a half-open base interval; the BED ranges for that contig, pre-sorted by start, are intersected against the node interval with an advancing range cursor, and the resulting (window, node) pairs are deduplicated. For each unique pair, the node length is added to the window denominator and to the numerator of every group in that node's CSR row, with atomics that are contention-free in the common case since distinct contigs map to disjoint window-index ranges. A node counts at most once per group, matching `odgi pav` semantics; the result is node-length-weighted PAV ratios computed without a whole-graph object.

Growth (path-iterative). This command computes Panacus-style union growth for nodes. Threads loop over groups under dynamic scheduling and reduce into a single shared coverage array $\texttt{cov}[v]$, where each node is counted at most once per group: a per-thread last-seen stamp—a rolling byte reset on wrap, which avoids a per-group clear of the stamp array—suppresses duplicate counts within a group, so a thread issues an atomic increment only on a node's first visit. The coverage vector is then reduced to a histogram $\texttt{hist}[c] = |\{v : \texttt{cov}[v] = c\}|$ by merging per-thread partials in a short critical section. For a pangenome with N groups, the expected number of nodes observed in a random subset of size k is

$$\texttt{growth}[k] = \sum_c \texttt{hist}[c] \left(1 - \frac{\binom{N-c}{k}}{\binom{N}{k}}\right),$$

where the ratio $\binom{N-c}{k}/\binom{N}{k}$ is evaluated as the stable product $\prod_{i=0}^{c-1}(N-k-i)/(N-i)$ rather than by forming the binomial coefficients, which would overflow at pangenome N; the curve is evaluated in parallel over cohort sizes k. The accumulator is linear in the number of nodes, and the fold matches Panacus union-growth semantics (count = node, coverage ≥ 1, quorum ≥ 0).

Similarity, depth, and stats. The remaining three reuse the substrate without new accumulator structure. `similarity` generalizes `pav`'s node-join from reference windows to *all group pairs*, reproducing the `odgi similarity` matrix: a first pass (parallel over slices) tallies per-node visit multiplicity, a second (parallel over groups) folds these into per-group coverage and flattens them into the node→groups CSR, and a join parallel over nodes accumulates, for each node, the minimum of every covering pair's coverages—via atomic updates—into a dense upper-triangular intersection matrix whose diagonal holds each group's total length; GFAz reports Jaccard, cosine, Dice, and estimated-identity columns identical to `odgi`, performing the arithmetic in double precision to avoid the integer overflow `odgi` can incur at genome-scale group lengths. `depth` reproduces `odgi depth` in a single streaming pass parallel over slices: a shared per-node counter accumulates total visits with atomic increments, while a per-thread last-seen stamp also reports the number of distinct paths and walks touching each node without a per-thread copy of the node array, all in $O(\text{nodes})$ regardless of haplotype count. `stats` is the degenerate case—it reports graph dimensions (node, edge, path, and step counts and total length, matching `odgi stats`) directly from the container's metadata columns, decoding *no* traversals, with an optional A/C/G/T tally over the segment sequences.

5.8 Scope

These six commands delimit the intended scope of the compute engine: analytics whose hot path is traversal streaming plus a compact fold, spanning a zero-pass metadata read (`stats`) to a topology-aware variant caller (`deconstruct`). Operations that require arbitrary node-to-step queries, graph editing, or full topology transformation remain better served by materialized graph toolkits.

6 Evaluation Results

We evaluate the three headline analyses of Section 5 against the standard tool for each task: `deconstruct` versus `vg deconstruct` [5], `growth` versus Panacus [18], and `pav` versus `odgi pav` [7]. All runs were performed on a single node with an AMD Ryzen Threadripper PRO 9955WX (16 physical cores, 32 hardware threads) and 503 GiB of RAM. Each command was run at **1** and **16** threads, one at a time, and wrapped in `/usr/bin/time -v` to record wall-clock time and peak resident set size (RSS). GFAz reads the compressed `.gfaz` container directly; the baselines read the uncompressed GFA (`odgi` additionally builds an `.og` index first, timed separately).

The chr1 and chr6 graphs are human pangenome graphs with a linear CHM13 reference, used for `deconstruct` and `pav`; the *E. coli* graph is a 13-strain bacterial pangenome used for `growth`. (The bacterial graph is circular and stores only W-line walks, so it is not a meaningful target for reference-anchored `deconstruct` nor for `odgi pav`, which does not load W-line paths; it is retained as a `growth` data point.)

6.1 Deconstruct (GFA → VCF)

Table 15 reports the GFA → VCF deconstruction of the chr1 and chr6 graphs against `vg deconstruct`. GFAz produces the same set of variant sites as `vg`—the record counts agree to within 0.13% (chr1: 1,593,899 vs. 1,593,956; chr6: 1,304,121 vs. 1,302,385), consistent with the ∼99.99% position concordance established in Section 5.6—while running 36–71× faster and using 3.3–3.4× less peak memory. The output is thread-count invariant: the 1- and 16-thread runs emit byte-identical VCFs. Because deconstruction must additionally hold the per-site allele sequences in memory, GFAz's peak RSS here is comparable to the size of the uncompressed GFA rather than below it; the lighter analyses in Sections 6.2–6.3 stay well under it.

Scaling to whole human pangenomes. Table 16 reports `deconstruct` on the three whole-genome HPRC graphs, deconstructing every CHM13 chromosome. Here GFAz runs *alone*: `vg` must load the entire uncompressed graph and does not complete on these inputs within practical memory, whereas GFAz operates on the 2.2–4.8 GB container. On the largest graphs (358 and 369 GB of GFA) GFAz emits the genome-wide VCF (∼35 M records) in roughly 40 minutes at 16 threads, with peak RSS of 191–195 GB—about half the size of the GFA the input was decompressed from, and well within a single node. On hprc-v1.1, the 1- vs. 16-thread runs (79.9 min vs. 5.1 min) show 15.6× scaling; single-threaded runs on the two largest graphs are omitted as impractical (>10 h extrapolated).

6.2 Pangenome Growth

Table 17 compares `growth` against Panacus, the state-of-the-art exact growth-curve tool, on all three graphs. With GFAz configured

Table 13: The six compute-engine commands, all built on one streaming decode substrate. "Resident state" excludes the grammar-compressed traversal store common to every command; node counts dominate group and window counts at pangenome scale.

Command	Reference tool	Accumulator	Traversal passes	Resident state	Regime
`deconstruct`	`vg deconstruct`	per-site allele table (+ segment DNA)	1 (+ topology pass)	$O(\text{sites})$ + DNA	path-iterative
`growth`	Panacus	coverage vector → histogram	1	$O(\text{nodes})$	path-iterative
`depth`	`odgi depth`	per-node visit counters	1	$O(\text{nodes})$	path-iterative
`stats`	`odgi stats`	metadata totals (opt. base tally)	0	$O(1)$	path-iterative
`pav`	`odgi pav`	node→groups CSR + matrix	3	$O(\text{nodes + win.×grp.})$	node-join
`similarity`	`odgi similarity`	node→groups CSR + matrix	2 (+ join)	$O(\text{nodes + grp.}^2)$	node-join

Table 14: Evaluation graphs. GFAz operates on the compressed container, which is 18–35× smaller than the GFA the baselines must read.

Graph	GFA	`.gfaz`	Ratio
chr1 (HPRC, smoothed)	7.58 GB	214 MB	35.4×
chr6 (HPRC, smoothed)	4.06 GB	115 MB	35.4×
E. coli (MGC)	118 MB	6.4 MB	18.4×

Table 15: `deconstruct` vs. `vg deconstruct`: wall-clock time and peak RSS at 1 and 16 threads. GFAz reads the 35×-smaller `.gfaz`; `vg` reads the GFA.

		1 thread		16 threads	
Graph	Tool	Time	RSS	Time	RSS
chr1	vg	1245 s	30.1 GB	805 s	30.3 GB
	GFAz	34.3 s	8.8 GB	11.3 s	8.3 GB
chr6	vg	898 s	18.4 GB	349 s	18.4 GB
	GFAz	21.9 s	5.7 GB	7.0 s	5.2 GB

Table 16: `deconstruct` on whole-genome HPRC graphs (GFAz only; `vg` does not complete in practical memory). Time and peak RSS; "GFA" is the size of the uncompressed input GFAz never materializes.

		1 thread		16 threads	
Graph (GFA)	Records	Time	RSS	Time	RSS
hprc-v1.1 (48 GB)	23.1 M	79.9 min	36.8 GB	5.1 min	37.4 GB
hprc-v2.0 (358 GB)	35.5 M	—	—	39.8 min	191 GB
hprc-v2.1 (369 GB)	34.8 M	—	—	41.4 min	195 GB

to Panacus's grouping (`sample-hap-seq`), the two tools produce the *same* growth curve: at every growth point the GFAz expectation matches Panacus's reported value (e.g. chr1 at $k = 100$: 5,655,489; at $k = 1000$: 10,927,766; identical full-pangenome totals of 13,590,187, 4,781,777, and 819,657 nodes for chr1, chr6, and *E. coli*). GFAz computes this curve 15–25× faster while using 9–16× less peak memory. Crucially, its peak RSS stays at a few hundred megabytes—more than 10× below the size of the uncompressed GFA—directly demonstrating the sub-GFA memory thesis: the traversal store remains grammar-compressed and only a compact coverage accumulator is added.

Scaling to whole human pangenomes. Table 18 reports `growth` on the whole-genome HPRC graphs. Panacus completes on all three,

Table 17: `growth` vs. Panacus: wall-clock time and peak RSS at 1 and 16 threads. Both tools report identical growth curves.

		1 thread		16 threads	
Graph	Tool	Time	RSS	Time	RSS
chr1	Panacus	109.9 s	6.13 GB	18.3 s	6.16 GB
	GFAz	7.3 s	0.46 GB	0.74 s	0.66 GB
chr6	Panacus	57.3 s	3.75 GB	9.0 s	3.77 GB
	GFAz	3.4 s	0.24 GB	0.36 s	0.31 GB
E. coli	Panacus	0.98 s	0.13 GB	0.89 s	0.13 GB
	GFAz	0.07 s	0.02 GB	0.01 s	0.03 GB

Table 18: `growth` on whole-genome HPRC graphs. Panacus completes on all three but is 230–369× slower and uses 7–25× more memory; single-threaded Panacus was not run. Time and peak RSS.

		1 thread		16 threads	
Graph (GFA)	Tool	Time	RSS	Time	RSS
hprc-v1.1 (48 GB)	Panacus	—	—	23.8 min	45.8 GB
	GFAz	69.2 s	4.9 GB	6.2 s	6.3 GB
hprc-v2.0 (358 GB)	Panacus	—	—	245 min	327 GB
	GFAz	—	—	39.8 s	12.9 GB
hprc-v2.1 (369 GB)	Panacus	—	—	251 min	336 GB
	GFAz	—	—	41.2 s	13.7 GB

but its runtime and memory track the uncompressed GFA: 23.8 min / 45.8 GB on hprc-v1.1 and over four hours with 327–336 GB of RSS on the two largest graphs. GFAz computes the identical curve—matching Panacus's node totals exactly at every growth point—in 6.2–41 s with only 6.3–13.7 GB of RSS, a 230–369× speedup with 7–25× less memory and peak RSS about 27× below the GFA. On hprc-v1.1 the GFAz 1- vs. 16-thread runs (69.2 s vs. 6.2 s) give 11× scaling.

6.3 Presence/Absence Variation

Table 19 compares `pav` against `odgi pav` on the chr1 and chr6 graphs, computing the whole-path PAV matrix (2,262 and 1,410 reference windows × 46 sample groups). Both tools emit a matrix with identical structure; GFAz's PAV ratios reproduce `odgi`'s reference semantics (Section 5.7). The performance gap is the largest of the three analyses: at 16 threads GFAz is 299× (chr1) and 613× (chr6) faster than `odgi pav` while using 3.1–3.3× less peak memory, and it operates directly on the `.gfaz`, avoiding the `odgi build` step (a further 61 s / 31 s) that `odgi pav` requires. Single-threaded `odgi`

Table 19: `pav` vs. `odgi pav`: wall-clock time and peak RSS. `odgi` additionally requires an `odgi build` step (chr1 61 s, chr6 31 s at 16 threads); single-threaded `odgi pav` did not complete (>1 day).

		1 thread		16 threads	
Graph	Tool	Time	RSS	Time	RSS
chr1	`odgi`	—†	—	3499 s	31.9 GB
	GFAz	106 s	8.7 GB	11.7 s	9.5 GB
chr6	`odgi`	—†	—	4759 s	19.4 GB
	GFAz	72 s	5.8 GB	7.8 s	6.4 GB

† single-threaded `odgi pav` did not complete (>1 day).

`pav` did not finish within a day and is omitted; even at 16 (and, in prior runs, 32) threads it remains three orders of magnitude slower than GFAz, whose single-threaded run already completes in under two minutes.

Scaling to whole human pangenomes. On hprc-v1.1, `pav` over all 30,640 whole-walk windows completes in 1.8 min at 16 threads (19.3 min at 1 thread, 10.7× scaling) with 77 GB peak RSS. Unlike `deconstruct` and `growth`, `pav` does *not* keep its working set below the GFA: the PAV node-set index and per-window accumulators are not grammar-compressed, so memory grows with the number of windows and nodes. On the two largest graphs (94,554 and 60,807 windows over 148–166 M nodes) this working set exceeds the 503 GB node—both runs were terminated near 470 GB before completing. This is the one analysis that does not yet meet the sub-GFA memory goal at whole-genome scale; compressing the PAV index is the most pressing optimization (its working set, not the traversal store, is the bottleneck).

6.4 A Second Pangenome Family: HGSVC3

All graphs above are drawn from the HPRC. To test that the compute engine is not specialized to a single graph-construction pipeline, we add HGSVC3—a whole-genome Minigraph–Cactus pangenome from a different consortium (the Human Genome Structural Variation Consortium), spanning 65 samples plus the CHM13 and GRCh38 references (29,130 walks, 107.1 M nodes). Its GFA is 80.1 GB; GFAz compresses it to a 2.85 GB container (28.1×), and the engine runs all three analyses directly on that container. Table 20 reports the result against `vg deconstruct` and Panacus. The behavior matches the HPRC graphs closely. On `growth`, GFAz reproduces Panacus's curve exactly—identical node totals at every growth point (e.g. k=100: 59,118,657; k=1000: 85,744,210; full pangenome 107,125,789)—while running 253× faster (9.6 s vs. 40.6 min) with 8.8× less memory. On `deconstruct`, GFAz agrees with `vg` to within 0.17% on record count (25,323,695 vs. 25,368,045) while running 15.2× faster; the memory gap is the striking figure here—`vg` needs 397 GB, nearly the entire 503 GB node, whereas GFAz uses 51.9 GB (7.6× less). Thread scaling is 10.7–14.6× across the three analyses, and—as on the HPRC giants—`pav` is the lone analysis whose uncompressed working set (126 GB) exceeds the input GFA. The engine thus generalizes across pangenome families without modification.

Table 20: GFAz compute engine on HGSVC3, a non-HPRC whole-genome pangenome (80.1 GB GFA, 2.85 GB `.gfaz`), vs. `vg deconstruct` and Panacus. Wall-clock time and peak RSS at 1 and 16 threads; single-threaded baselines were not run (>2 h each at 16 threads). `pav` is reported GFAz-only (`odgi` does not load the W-line walks).

		1 thread		16 threads	
Analysis	Tool	Time	RSS	Time	RSS
`deconstruct`	`vg`	—	—	132.9 min	396.9 GB
	GFAz	127.6 min	51.7 GB	8.7 min	51.9 GB
`growth`	Panacus	—	—	40.6 min	71.1 GB
	GFAz	102.9 s	6.5 GB	9.6 s	8.1 GB
`pav`	GFAz	42.4 min	126.2 GB	3.6 min	126.2 GB

Summary

Across all three analyses, computing directly on the compressed container makes GFAz one to three orders of magnitude faster than the established tools—15–71× over `vg deconstruct`, 15–369× over Panacus, and 299–613× over `odgi pav`—and, for `deconstruct` and `growth`, keeps peak RSS below the size of the uncompressed GFA. These gains hold across pangenome families: on HGSVC3, a whole-genome graph from a different consortium, GFAz reproduces `vg` and Panacus outputs while running 15–253× faster with 7.6–8.8× less memory—`vg deconstruct` alone needs 397 GB, nearly the entire node. The same property lets GFAz scale to whole human pangenomes the baselines cannot practically handle: on the 358 and 369 GB HPRC graphs, `vg` and `odgi` cannot load the input within memory, while Panacus `growth` completes only after ~4 hours and ~330 GB—whereas GFAz runs `deconstruct` and `growth` from a 4.5 GB container in tens of seconds to tens of minutes with an order of magnitude less memory. The lone exception is whole-genome `pav`, whose uncompressed working set still exceeds node memory on the two largest graphs—the clearest target for the next round of optimization.

7 Conclusion and Future Work

We presented GFAz, a high-performance GFA compressor that combines orientation-aware 2-mer grammar compression with a columnar, type-specific encoding of all GFA record types. By exploiting the canonical symmetry of forward and reverse node orientations, GFAz collapses traversal redundancy in linear time across iterative grammar rounds. Fully parallel CPU (OpenMP) and GPU (CUDA/nvComp) backends deliver state-of-the-art compression ratios (20–80×) and throughput on datasets ranging from microbial pangenomes to whole-genome human graphs.

Our current GPU backend requires the full traversal stream to fit in device memory, which prevents processing the largest graphs (300+ GB) on current hardware. In future work, we plan to introduce memory-aware scheduling for both CPU and GPU pipelines by partitioning traversals into streaming tiles that can be compressed and decompressed independently within a fixed memory budget to eliminate out-of-memory failures regardless of input size.

Acknowledgments

We would like to acknowledge the Human Pangenome Reference Consortium (BioProject ID: PRJNA730823) and its funder, the National Human Genome Research Institute (NHGRI). This work was partially supported by the National Institutes of Health (NIH) under Award Number R01GM157443.

References

[1] James K Bonfield. 2022. CRAM 3.1: advances in the CRAM file format. *Bioinformatics* 38, 6 (2022), 1497–1503.
[2] Yann Collet. 2021. RFC 8878: Zstandard Compression and the'application/zstd'Media Type.
[3] Jordan M Eizenga, Adam M Novak, Jonas A Sibbesen, Simon Heumos, Ali Ghaffaari, Glenn Hickey, Xian Chang, Josiah D Seaman, Robin Rounthwaite, Jana Ebler, et al. 2020. Pangenome graphs. *Annual review of genomics and human genetics* 21, 1 (2020), 139–162.
[4] Erik Garrison, Andrea Guarracino, Simon Heumos, Flavia Villani, Zhigui Bao, Lorenzo Tattini, Jörg Hagmann, Sebastian Vorbrugg, Santiago Marco-Sola, Christian Kubica, et al. 2024. Building pangenome graphs. *Nature Methods* 21, 11 (2024), 2008–2012.
[5] Erik Garrison, Jouni Sirén, Adam M Novak, Glenn Hickey, Jordan M Eizenga, Eric T Dawson, William Jones, Shilpa Garg, Charles Markello, Michael F Lin, et al. 2018. Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nature biotechnology* 36, 9 (2018), 875–879.
[6] GFA Specification Working Group. 2023. GFA1: Graphical Fragment Assembly Format Specification. https://gfa-spec.github.io/GFA-spec/GFA1.html. Accessed: 2026-02-08.
[7] Andrea Guarracino, Simon Heumos, Sven Nahnsen, Pjotr Prins, and Erik Garrison. 2022. ODGI: understanding pangenome graphs. *Bioinformatics* 38, 13 (2022), 3319–3326.
[8] Peter Heringer and Daniel Doerr. 2025. Human readable compression of GFA paths using grammar-based code. *bioRxiv* (2025), 2025–05.
[9] Glenn Hickey, David Heller, Jean Monlong, Jonas A. Sibbesen, Jouni Sirén, Jordan Eizenga, Eric T. Dawson, Erik Garrison, Adam M. Novak, and Benedict Paten. 2020. Genotyping structural variants in pangenome graphs using the vg toolkit. *Genome Biology* 21, 1 (2020), 35. doi:10.1186/s13059-020-1941-7
[10] Glenn Hickey, Jean Monlong, Jana Ebler, Adam M Novak, Jordan M Eizenga, Yan Gao, Tobias Marschall, Heng Li, and Benedict Paten. 2024. Pangenome graph construction from genome alignments with Minigraph-Cactus. *Nature biotechnology* 42, 4 (2024), 663–673.
[11] Human Pangenome Reference Consortium. 2025. Human Pangenome Project Data Portal. https://42basepairs.com/browse/s3/human-pangenomics/pangenomes. Public HPRC pangenome graph datasets hosted via the 42basepairs platform; accessed February 8, 2026.
[12] Human Pangenome Reference Consortium. 2025. Human Pangenome Reference Consortium: Release Timeline. https://humanpangenome.org/release-timeline/. Accessed: February 8, 2026.
[13] Heng Li, Bob Handsaker, Alec Wysoker, Tim Fennell, Jue Ruan, Nils Homer, Gabor Marth, Goncalo Abecasis, Richard Durbin, and 1000 Genome Project Data Processing Subgroup. 2009. The sequence alignment/map format and SAMtools. *bioinformatics* 25, 16 (2009), 2078–2079.
[14] Wen-Wei Liao, Mobin Asri, Jana Ebler, Daniel Doerr, Marina Haukness, Glenn Hickey, Shuangjia Lu, Julian K Lucas, Jean Monlong, Haley J Abel, et al. 2023. A draft human pangenome reference. *Nature* 617, 7960 (2023), 312–324.
[15] Mao-Jan Lin, Sheila Iyer, Nae-Chyun Chen, and Ben Langmead. 2024. Measuring, visualizing, and diagnosing reference bias with biastools. *Genome Biology* 25, 1 (2024), 101.
[16] Nina Marthe and Francois Sabot. 2025. Data used for the analysis described in GrAnnoT paper. doi:10.23708/DO1RTF
[17] NVIDIA Corporation. 2026. NVIDIA/nvcomp: Repository for nvCOMP docs and examples. https://github.com/NVIDIA/nvcomp. GitHub repository, accessed February 9, 2026.
[18] Luca Parmigiani, Lars Wienbrandt, and Tobias Marschall. 2024. Panacus: fast and exact pangenome growth and core size estimation. *Bioinformatics* 40, 12 (2024), btae720. doi:10.1093/bioinformatics/btae720
[19] Benedict Paten, Jordan M. Eizenga, Yohei M. Rosen, Adam M. Novak, Erik Garrison, and Glenn Hickey. 2018. Superbubbles, Ultrabubbles, and Cacti. *Journal of Computational Biology* 25, 7 (2018), 649–663. doi:10.1089/cmb.2017.0251
[20] Simona Secomandi, Guido Roberto Gallo, Riccardo Rossi, Carlos Rodríguez Fernandes, Erich D Jarvis, Andrea Bonisoli-Alquati, Luca Gianfranceschi, and Giulio Formenti. 2025. Pangenome graphs and their applications in biodiversity genomics. *Nature Genetics* 57, 1 (2025), 13–26.
[21] Jouni Sirén, Erik Garrison, Adam M Novak, Benedict Paten, and Richard Durbin. 2020. Haplotype-aware graph indexes. *Bioinformatics* 36, 2 (2020), 400–407.
[22] Jouni Sirén, Jean Monlong, Xian Chang, Adam M Novak, Jordan M Eizenga, Charles Markello, Jonas A Sibbesen, Glenn Hickey, Pi-Chuan Chang, Andrew Carroll, et al. 2021. Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science* 374, 6574 (2021), abg8871.
[23] Jouni Sirén and Benedict Paten. 2022. GBZ file format for pangenome graphs. *Bioinformatics* 38, 22 (2022), 5012–5018.
[24] Ting Wang, Lucinda Antonacci-Fulton, Kerstin Howe, Heather A Lawson, Julian K Lucas, Adam M Phillippy, Alice B Popejoy, Mobin Asri, Caryn Carson, Mark JP Chaisson, et al. 2022. The Human Pangenome Project: a global resource to map genomic diversity. *Nature* 604, 7906 (2022), 437–446.
[25] Ryan R. Wick, Mark B. Schultz, Justin Zobel, and Kathryn E. Holt. 2015. Bandage: Interactive visualization of de novo genome assemblies. *Bioinformatics* 31, 20 (2015), 3350–3352. doi:10.1093/bioinformatics/btv383
[26] Taolue Yang, Youyuan Liu, Bo Jiang, and Sian Jin. 2024. GPUFASTQLZ: An Ultra Fast Compression Methodology for Fastq Sequence Data on GPUs. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 317–325.
[27] Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. 2023. A formal perspective on byte-pair encoding. In *Findings of the Association for Computational Linguistics: ACL 2023*. 598–614.